LLM-Based Test Oracles: Source-of-Authority Taxonomy—A Systematic Literature Review

Ali Hassaan Mughal, *Graduate Student Member, IEEE,* and Muhammad Bilal

arXiv:2607.05031v3 [cs.SE] 2 Sep 2026

***Abstract*—Large language models (LLMs) increasingly decide whether software behaves correctly, either by writing a test oracle or by acting as one. Yet two oracles can look identical and rest on different ground: one assertion encodes a written specification, another only what the model learned in training. Prior secondary studies sort oracles by form or by technique, rarely by the property that governs how far a verdict can be trusted: where its authority comes from. This systematic literature review, reported under the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 guidelines, screens 2,436 records to 54 included studies, extended by citation searching (snowballing) to 83 in total. We read the corpus along three axes: the source of an oracle's authority, the form it takes, and the mechanism that adjudicates it. Just over half of the corpus reaches a verdict with no specification at all. That is what lets these oracles work on code with no specification to consult, and what leaves a challenged verdict with less to fall back on. Source and mechanism cross-cut rather than coincide, so a label such as LLM-as-a-judge names how a verdict is produced, not why it should be trusted. Oracle quality is most often judged by resemblance to a known oracle rather than by whether injected faults are caught. The first question to ask of any LLM oracle is therefore what one would point to in defending its verdict. The protocol, search query, and per-study coding sheet are released.**

***Index Terms*—Large language models, LLM-as-a-judge, oracle problem, software quality assurance, software testing, systematic literature review, test automation, test oracle.**

I. Introduction

SOFTWARE now runs in almost every part of daily life, and how much we can trust it depends largely on how well we test it. A test feeds an input to a program, runs it, and then asks one question: is the result correct? The part of the test that answers that question is the *test oracle*. A test is only as good as its oracle.

Deciding what counts as correct is hard. For many programs there is no easy way to know the right answer in advance. This difficulty has a name, the *oracle problem*. Barr et al. [1] surveyed it before large language models (LLMs) were in wide use. They organized the known ways to obtain an oracle and showed how often a strong oracle is simply missing.

LLMs have changed the picture. An LLM can now write an oracle for a program [2]–[4], or act as the oracle itself by judging the output directly [5]–[7]. This is a fast-growing area [8], [9]: most of the reviewed studies appeared in 2025 or later.

The literature describes these new oracles. It characterizes them, however, by a feature that does not, by itself, capture how far to trust the verdict. Papers sort oracles by their shape, such as an assertion [3] or a metamorphic relation [10], or by the technique used, such as "LLM as a judge" [5], [7]. They rarely ask where the verdict's authority comes from, yet that is what sets the ceiling on how much to trust it. Two oracles can look identical yet rest on different ground. One assertion may encode a written specification; another, with the same syntax, may encode only what the model guessed.

This review organizes the literature by that missing feature: the *source of authority*, where the correctness of the verdict ultimately comes from. We make three contributions:

- a taxonomy of oracle sources that extends Barr et al. into the LLM era, kept separate from two questions the literature mixes in: the *form* of the oracle and the *mechanism* that computes pass or fail;
- a systematic literature review (SLR), conducted and reported under the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 guidelines [11], that screens 2,436 records down to 54 included studies, extended by a citation-searching (snowballing) round that adds 29 further studies, for 83 in total;
- a synthesis of what the lens reveals: specification independence follows almost entirely from the source; source and mechanism cross-cut rather than coincide; one source category goes unoccupied as a primary source of LLM-supplied oracles; and the most frequently reported risks are weak oracles and model hallucination.

II. Background: The Oracle Problem in Plain Terms

A. What a test oracle is

Every test has four parts. There is an *input*; the *program under test*, often called the system under test; the *output* the program produces; and the *oracle*, the part that decides whether the output is correct (Fig. 1). The first three parts are usually easy to set up. The oracle is the difficult one.

We will use one small function as a running example through the whole article: `isLeapYear`, which should return true when a year is a leap year. A year is a leap year if it divides by 4. The exception is years that divide by 100, which are not leap years, unless they also divide by 400. So 2024 is a leap year, 1900 is not, and 2000 is. To test the function

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3729738.

This work received no external funding. The authors declare no conflicts of interest.

A. H. Mughal is an Independent Researcher, USA (e-mail: alihassaan-mughal.work@gmail.com).

M. Bilal is with the Technical University of Munich, Germany (e-mail: m.bilal@tum.de). He is the corresponding author.

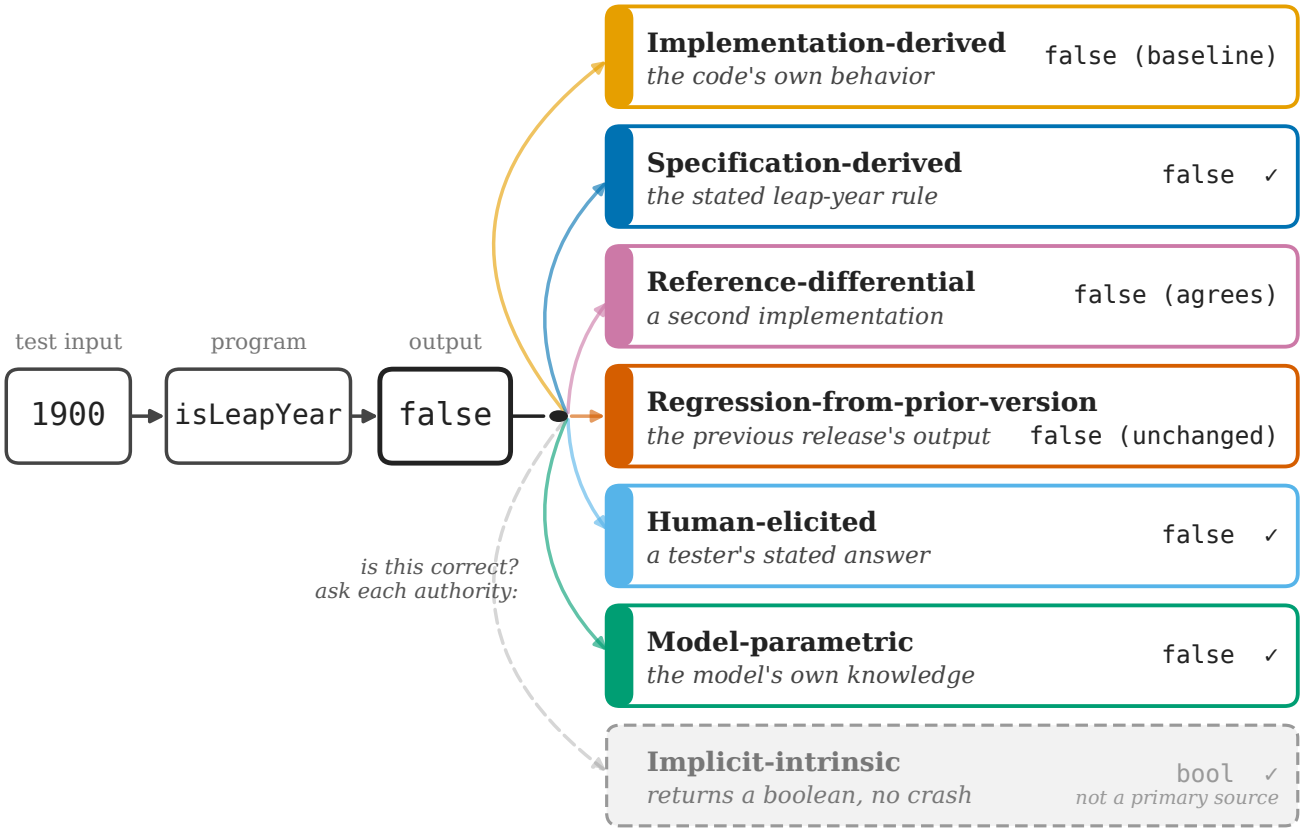


Fig. 1. The seven sources of oracle authority, illustrated on `isLeapYear(1900)`. The grey dashed box marks the one category no included study uses as a *primary* source (it appears as a secondary source in four systems).

we can run it on the input 1900. Suppose it returns false. Is that correct? Answering that is the oracle's job. To answer it, the oracle must know from somewhere that 1900 is not a leap year.

B. Why it is hard

That "from somewhere" is the core difficulty. For `isLeapYear` the right answers are known, so an oracle is easy. For most real programs they are not known in advance and are costly to work out by hand. This is the *oracle problem*: the difficulty of obtaining a reliable way to judge whether an output is correct.

Over the years, testers have found partial answers that avoid needing the exact right answer for every input. An *assertion* [12] states a condition the output must meet, such as that a year reported as a leap year is divisible by 4. A *metamorphic relation* [13] compares two related runs instead of checking one output against a known value. For example, running the function twice on the same year should give the same result. *Differential testing* [14] runs two programs that should agree and treats any disagreement as a possible bug. Each of these gives a verdict without a full description of correct behavior. Barr et al. [1] surveyed these and other techniques in detail.

C. What LLMs add

An LLM can take part in the oracle in two ways. It can *generate* an oracle, for example by reading a function's documentation and writing the assertions that check it [4], [15], [16]. Or it can *be* the oracle, by looking at an output and judging it directly, the way a human reviewer would [5], [17], [18]. Both were impractical at scale before LLMs, and both are now common. This raises a new question, the subject of this review: when an LLM writes the oracle, or makes the verdict itself, where does the authority for that verdict come from?

III. The Source-of-Authority Framework

The central idea of our framework is to separate three questions that the literature usually treats as one.

A. Three distinct questions

When an LLM-based oracle gives a verdict, we can ask three things about it.

1) **Source of authority.** Where does the correctness of the verdict ultimately come from? By *authority* we mean the artifact or agent whose word makes the verdict count as correct: what one would point to when defending a pass or a fail.

2) **Form.** What shape does the oracle take, for example an assertion or a metamorphic relation?
3) **Mechanism.** How is the pass-or-fail decision computed, for example by running an assertion or by asking the model to judge?

Most prior work answers only the second or third question. The first sets the ceiling on trust: an oracle is only as trustworthy as the source its authority rests on. Trust here means evidential defensibility, the independent ground a verdict can appeal to, rather than a measured error rate; the ordering of sources is a conceptual claim, not an empirical result of this review. The ceiling is also set by the source's kind, not the quality of the individual artifact: a defective specification can in practice rank below a well-validated reference implementation.

B. Seven sources of authority

The framework distinguishes seven sources of authority (Fig. 1). The same program output can be judged by any of them, each drawing on a different authority.

In words: an *implementation-derived* oracle takes the code, or the tests already written for it, as the authority. It accepts what the code currently does, so it catches later changes but not a bug that is already present. A *specification-derived* oracle takes an external description of intended behavior as the authority. The description can be formal, such as a logical postcondition; semi-formal, such as a structured rule or a Gherkin scenario; or natural-language, such as a docstring or a requirement. A *reference-differential* oracle takes a separate implementation as the authority and flags disagreement. A recent variant uses several implementations the LLM writes itself and treats their agreement as the expected answer [19], [20]. The strength of this source rests on how independent the reference is: an independently developed reference implementation, a differential comparison among existing implementations, and a consensus among implementations the same model generates carry successively weaker independence assumptions. The remaining four are a *model-parametric* oracle (the model's own knowledge, with no external reference), a *human-elicited* oracle (criteria a person supplies), a *regression-from-prior-version* oracle (an earlier version as the reference), and an *implicit-intrinsic* oracle (crashes or well-formedness), each defined in Fig. 1.

Table I locates the seven sources against Barr et al.'s oracle taxonomy [1]: five categories are retained or renamed from established oracle classes, one is subdivided, and one, model-parametric authority, is newly introduced. What prior classifications could not express is not the individual categories but the axis itself: they organize oracles by how the artifact is obtained or expressed, not by whose authority the verdict rests on, and it is the latter that the LLM era makes load-bearing.

TABLE I
THE SEVEN SOURCES OF AUTHORITY AGAINST BARR ET AL.'S ORACLE TAXONOMY [1].

This review	Barr et al.	Relation
Specification-derived	Specified oracles	Retained; widened to semi-formal and natural-language specifications
Implementation-derived	Derived oracles	Subdivided: restricted to artifacts of the current implementation
Reference-differential	Pseudo-oracles (a derived subtype)	Renamed; graded by reference independence
Regression-from-prior-version	Derived from previous versions	Retained
Implicit-intrinsic	Implicit oracles	Renamed
Human-elicited	Human oracle	Retained; sharpened to criteria a person supplies
Model-parametric	(no counterpart)	Newly introduced: the model's parametric knowledge as the authority

C. Specification dependence

The seven sources split by one question: does the oracle need an external specification? In the idealized split, only specification-derived oracles do; the other six are defined to reach a verdict without one. Section V examines how closely the corpus follows this split, including the few boundary cases that cross it.

Because every LLM-based oracle is driven by a prompt, it is worth fixing what counts as an external specification here. A prompt's task instruction, such as *decide whether this output is correct* or *write assertions for this method*, is not a specification: it directs the oracle without describing the program's intended behavior. A specification is an explicit, auditable description of that intended behavior, such as a docstring, a requirement, an application programming interface (API) contract, or a structured rule; when the verdict rests on such an artifact the oracle is specification-derived, regardless of the mechanism that consumes it and regardless of whether the artifact was supplied to the oracle or, in one boundary case, synthesized by the model itself into an executable relation (Section V). What separates this from a model-parametric oracle is auditability, not authorship: when the description of intended behavior is left implicit in the model's weights and never externalized into an inspectable artifact, the oracle is model-parametric. Whether an external specification *pre-existed*, as opposed to being synthesized on the fly, is the separate axis of specification dependence. Specification dependence is therefore a property of what the oracle is given and produces, not of whether a prompt is present. The channel through which a specification reaches the oracle, whether a system instruction, the user turn, retrieved context, or a file the oracle reads, likewise does not change the coding: what matters is that an auditable description of intended behavior is present, not how it arrived.

Two further boundaries need fixing, because the same artifact can be described from either side. Human-elicited against specification-derived turns on the artifact the verdict appeals to, not on who stands behind it: a durable, inspectable description of intended behavior is a specification whoever wrote it, while a criterion a person supplies for this judgment alone, such as a rubric written for the run or a reviewer's own decision, is human-elicited. Implementation-derived against specification-derived turns on how an existing test suite is used. We code existing tests as implementation-derived, be-

cause the reviewed systems read them as a record of what the code currently does, so the oracle inherits any fault the tests already encode. That is a coding rule and not a claim that tests never carry independent requirements: where a study rests its tests on an external requirement the coding records a secondary specification-derived source, as it does for two of the implementation-derived studies [60], [96]. Each rule is applied through the anchoring quotation recorded for the study, so a reader who would draw the line elsewhere can see which sentence the coding rests on.

D. Source is not mechanism

Source and mechanism cross-cut: a single source can be checked by many mechanisms, and a single mechanism can serve many sources. A specification-derived oracle, for example, can be checked by running an assertion, by matching against an expected value, or by asking the model to judge. "LLM as a judge" is a mechanism, not a source: the judge may be leaning on a specification, on the model's own knowledge, or on criteria a person gave it. A few mechanisms recur across different sources (Fig. 9).

IV. Review Method

This SLR was planned, conducted, and reported under the PRISMA 2020 statement [11]. The protocol, the frozen search query, the operational criteria, and the per-record screening log are released as supplementary material.

A. Objective

The review maps and analyzes the literature on LLM-based test oracles according to the *source* from which each oracle draws its verdict authority. A study is within scope if it (i) produces a test oracle, or (ii) empirically studies how the oracle is determined, within a pipeline that uses an LLM. Pre-2021 foundational work is retained as background through citation only and is excluded from the primary mapping counts. The design is aligned with the established methodology for SLRs in software engineering of Kitchenham and Charters [21] in its use of a pre-specified protocol, criterion-based screening, and structured extraction; those guidelines informed how the review was structured rather than being implemented in full. PRISMA 2020 [11] supplies the reporting standard.

B. Research questions

The review is organized around three research questions (RQ1–RQ3): RQ1 maps the landscape of the literature, RQ2 characterizes the corpus along the three-axis taxonomy at the review's core, and RQ3 covers how the oracles are used and where they fall short. Within RQ2, the parts on the source of authority (RQ2.a) and the adjudication mechanism (RQ2.c) are the main focus.

RQ1 — The landscape.

RQ1 (Landscape): What software domains, systems under test, languages, models, and adaptation strategies characterize the LLM-based oracle literature?

RQ2 — The three-axis taxonomy. How does the corpus distribute across the source of the verdict's authority, the form the oracle takes, and the mechanism that adjudicates it? The three parts track the axes the framework keeps apart.

RQ2.a (Source of authority): From what source does each oracle draw the authority for its verdict, and how closely does the corpus follow the specification-dependence split those source definitions imply?

RQ2.b (Form): In what form is the oracle expressed (assertion, expected output value, metamorphic relation, invariant, property, classifier label, or exception oracle)?

RQ2.c (Adjudication mechanism and LLM role): How is the pass/fail computed, and what role does the LLM play in that computation?

RQ3 — Use and gaps. How are the oracles evaluated, how do they fail, and which regions of the taxonomy remain sparse or empty?

RQ3.a (Evaluation): How is each oracle evaluated, and how is the oracle's own correctness assessed?

RQ3.b (Failure modes): What failure modes are reported, and how are they mitigated?

RQ3.c (Gaps): Which regions of the source-by-mechanism space are sparse or empty?

A guiding hypothesis, examined in the synthesis, is that the source of authority (RQ2.a) cross-cuts the adjudication mechanism (RQ2.c): the two are conceptually distinct and not in one-to-one correspondence, rather than statistically independent. The same authority can be checked by different mechanisms, and a single mechanism such as LLM-as-judge can draw on different authorities.

C. Protocol and registration

The search query was frozen on 31 May 2026, and the inclusion and exclusion criteria were fixed in writing before screening began. In place of registration in a public registry, this pre-specified protocol is archived openly in the replication package [22].

D. Eligibility criteria

A record was *included* only if all of the following held:

I1 (Oracle focus): The work concerns a test oracle, that is, the mechanism that decides expected behavior or pass/fail (assertions, expected outputs, properties or invariants, metamorphic relations, differential comparison, or human judgment), rather than only test-input generation or coverage.

I2 (LLM involvement): An LLM generates, infers, or determines the oracle, or the study empirically examines how the oracle is determined or behaves in an LLM-based testing pipeline.

I3 (Window): Published between January 2021 and May 2026; for the citation-searching round, the window extends to that round's execution date (August 2026). Each arm's window ends on the date that arm was run. Forward citation searching necessarily reaches work published after

the database search was frozen, and holding that round to the earlier cut-off would discard the studies it was added to surface. Criterion I4 still applies, so these additions are peer reviewed like the rest of the corpus.

I4 (Form): At least four pages, written in English, with retrievable full text, peer reviewed and indexed (conference, journal, or early access).

A record was *excluded* if any of the following held:

E1: Testing *of* LLMs or natural language processing (NLP) systems (the LLM is the system under test, for example bias, hallucination, jailbreak, or robustness testing), rather than LLMs used *for* testing software.

E2: LLM test or code generation that does not foreground an oracle (input-only, coverage-only, prefix-only, or program-repair-only).

E3: Hardware-verification assertion or oracle generation, for example in SystemVerilog, register-transfer level (RTL), or very large-scale integration (VLSI). This is noted as an adjacent line of work in Section VIII.

E4: Secondary studies (surveys, systematic reviews, mapping studies). These are excluded from the primary set to avoid double counting, and are retained separately for the discussion of related reviews.

E5: Non-peer-reviewed preprints. These are deliberately excluded to keep peer review as an evidence-quality bar (criterion I4). The resulting under-coverage of preprint venues is reported as a threat to validity (Section VIII).

E6: Editorials, keynotes, posters, tool demonstrations, tutorials, extended abstracts, items under four pages, non-English items, and non-retrievable items.

E. Information sources

We searched three bibliographic databases under institutional subscription: Scopus, IEEE Xplore, and the ACM Digital Library. All searches were executed on 31 May 2026 over the field set title, abstract, and keywords, restricted to the publication window 2021–2026. Scopus indexes a large fraction of IEEE and ACM content and therefore also serves as a cross-database consolidation source.

F. Search strategy

The search combines a *population* concept (LLM terms) with a *task* concept (oracle and testing terms) using a Boolean `AND`. The frozen query is:

```
Concept A (LLM):
"large language model*" OR LLM OR LLMs OR GPT OR ChatGPT OR
      Codex OR Copilot
OR Llama OR "Code Llama" OR StarCoder OR Claude OR Gemini
     OR PaLM
OR "generative AI" OR "foundation model*" OR "code language
      model"

Concept B (oracle / test):
"test oracle*" OR "oracle generation" OR "oracle inference"
      OR "oracle problem"
OR "assertion generation" OR "test assertion*" OR "
     exception oracle"
OR "metamorphic testing" OR "metamorphic relation*" OR "
     property-based testing"
OR "differential testing" OR postcondition* OR precondition
     *
OR "user acceptance test*" OR "acceptance test*" OR Gherkin
OR "behavior-driven" OR "behaviour-driven" OR "behavior-
     driven development"

Search = Concept A AND Concept B
```

The query was rendered per database with identical Boolean logic and database-specific syntax, restricted to the computing subject area, without dropping any term, to remove cross-domain false positives such as the food-science and botany senses of "Codex" or "Gherkin". The exact per-database queries and the export procedure are in the supplementary material.

G. Selection process

Records exported from the three databases were merged and de-duplicated by normalized DOI, falling back to a normalized title when a DOI was absent. This left 2,245 unique records. Because the high-recall query admitted mostly off-topic records, the corpus was too large to screen every record twice by hand. We therefore screened in two stages, an arrangement disclosed in accordance with PRISMA 2020 item 8 on the use of automation tools. The first stage was a recall-oriented automated pre-filter that removes clearly out-of-scope records; the second was independent human dual screening that verifies the retained set in full and validates the pre-filter's exclusions on a stratified sample.

a) Stage 1: automated pre-filter: An LLM (Claude Opus 4.8) [23] classified every one of the 2,245 unique records against the eligibility criteria from title and abstract. It assigned each an include, maybe, or exclude decision and, where applicable, an exclusion reason code (E1–E6). Ambiguous records were assigned *maybe* and retained, resolving uncertainty toward full-text assessment. This stage retained 178 records and excluded 2,067. Each record was supplied to the model as its title and abstract together with the pre-registered inclusion and exclusion criteria, and the model returned an include, maybe, or exclude decision with an exclusion reason code (E1–E6), retaining any record it could not confidently exclude. Classification was run in June 2026 with the model's reasoning effort set to its maximum level. The provider's interface for this model does not expose sampling parameters such as temperature or top-p, so no decoding parameters were varied across records. The screening instruction applied criteria I1–I4 and E1–E6 to the title and abstract alone; the per-record decisions and reason codes, together with the exact screening instruction, are released for audit [22], and the recall and false-omission validation reported below (Table III) is the confusion-matrix summary of this stage.

b) Stage 2: independent human dual screening with validation: Two authors then screened a 385-record verification set, independently and blind both to the model's labels and to each other. The set comprised *all* 178 model-retained records plus a fixed-seed, reason-stratified 10% random sample of 207 records from the 2,067 model-excluded records. Screening all retained records catches every false positive; the exclusion sample estimates how often the model wrongly discards rel-

evant work. Each reviewer recorded an Include, Maybe, or Exclude decision with a one-line rationale for every record.

Inter-rater agreement was substantial, measured by Cohen's κ [24], a chance-corrected measure of how often two reviewers agree. On the operative retain-versus-exclude decision (Retain = Include ∪ Maybe), $\kappa = 0.79$ (95% bootstrap [25] confidence interval (CI) $[0.72, 0.85]$; 90.4% raw agreement). On the finer three-level scale it was $\kappa = 0.64$ (95% CI $[0.57, 0.71]$; 81.6% agreement). The 71 disagreements were reconciled conservatively. Include-versus-Maybe splits, where both reviewers leaned toward retention, were retained. The eight direct Include-versus-Exclude conflicts were decided individually against the full text. Exclude-versus-Maybe splits, where neither reviewer favored inclusion, were excluded. The retain-on-uncertainty principle stated above governs the recall-oriented pre-filter; this human stage is where precision is applied. Reconciliation yielded a consensus of 115 retained records (87 include, 28 maybe) and 270 excluded over the verification set.

c) Validation of the pre-filter: Treating the human consensus as the reference, the model achieved 99.1% recall on the verification set, agreeing with all but one consensus-retained record. That single miss was itself a borderline case reconciled to include only after full-text discussion. On the stratified sample of its own exclusions, the model's observed false-omission rate was $1/207 = 0.48\%$ (exact binomial, or Clopper–Pearson [26], 95% CI 0.01%–2.66%). If this rate held across the 1,860 model-excluded records not individually re-screened, it would imply roughly nine missed relevant records (the same interval scaled to the 1,860 unaudited exclusions gives roughly 0 to 50). Because the CI around a single observed omission is wide, this residual risk is carried as a threat to validity (Section VIII). The sampled exclusions and their decisions are also released for audit [22]. The model was otherwise over-inclusive, retaining 64 consensus-excluded records. Its precision is supplied by the human stage. Table IV sets out the confusion matrix these figures summarize.

H. Data extraction and coding

Each included study is coded against a 35-field codebook, fixed before extraction and released as supplementary material. The controlled vocabularies are likewise fixed in advance. The codebook is built on the three axes of Section III: the source of authority (RQ2.a), the form (RQ2.b), and the adjudication mechanism (RQ2.c). It also adds descriptive fields for the landscape, the evaluation, and the reported failure modes (RQ1, RQ3.a, RQ3.b). Source and mechanism are coded as two separate variables (Section III). A primary source and, where a system is compositional, a secondary source are recorded.

All 115 retained records, including the one later found to duplicate another, were coded against the locked codebook by the authors and then re-assessed in a second pass covering the two variables the review's conclusions rest on, eligibility and the primary source of authority, each supported by its own quotation drawn from the full text. The passes concurred on 80.7% of eligibility decisions (22 differences over the 114 unique full-text records, the duplicated pair counting once) and on 83.3% of primary-source codes (8 differences over the 48 records both passes judged includable; the remaining included studies were admitted only at adjudication, so the second pass assigned them no source). The 29 studies added by the citation-searching round were coded in two full passes over all fields, the second revising the primary source in four of the 29, a concordance of 86.2% (Table II). Two of those four revisions moved a study to model-parametric authority, one from implementation-derived and one from human-elicited, which is where the categories are hardest to hold apart in practice and where the boundary rules of Section III do most of their work. Every difference was adjudicated against the pre-registered boundary rules, the four hardest by a fresh full-text re-read; six eligibility decisions and two primary-source codes were resolved in favor of the first pass, and the two contested source codings [27], [28] are treated as boundary cases in Section VIII. Both coding passes and the per-record adjudication decisions are released [22], so the differences can be inspected or re-quantified independently. Each study's coding carries an anchoring quotation from its full text, so any coding can be re-traced and contested from the released per-study sheet. The synthesis is descriptive: it reports distributions across the taxonomy and reads gaps off the sparse or empty regions of the source-by-mechanism space. Effectiveness numbers are not pooled, because the studies differ too much in benchmark and metric: 64% use a custom benchmark. A lightweight, non-exclusionary quality appraisal of the included studies, scored on nine items derived from these coded fields, is reported in Section V-G.

TABLE II
WHERE THE TWO CODING PASSES PLACED EACH STUDY'S PRIMARY SOURCE OF AUTHORITY, FOR THE CITATION-SEARCHING ARM. ROWS ARE THE FIRST PASS, COLUMNS THE SECOND; THE DIAGONAL IS AGREEMENT. THE SAME MATRIX CANNOT BE SHOWN FOR THE DATABASE ARM, WHOSE FIRST PASS WAS REVISED IN PLACE WHEN THE ADJUDICATION WAS APPLIED. COLUMN ABBREVIATIONS FOLLOW TABLE XI.

1st↓ / 2nd→	Human	Impl	Model	Ref-diff	Regr	Spec	Total
Human	5	.	1	.	.	.	6
Impl	.	6	1	.	.	.	7
Model	.	.	6	.	.	.	6
Ref-diff	.	.	.	.	1	.	1
Regr	.	.	.	.	1	.	1
Spec	.	.	.	1	.	7	8
Total	5	6	8	1	2	7	29

I. Selection results

Figure 2 and Table III report the record flow and screening reliability. The searches identified 2,436 records, de-duplicated to 2,245. The dominant pre-filter exclusion reason was out-of-scope topical drift, records that matched the broad query terms but did not concern LLM-based software test oracles. In addition, a backward and forward citation-searching (snowballing) round over the 54 studies included from the database search, executed in August 2026 and screened against the same eligibility criteria, contributed 29 further included studies; the review therefore synthesizes 83 studies in total (Fig. 2).

TABLE III
SCREENING AND CODING RELIABILITY, AND PRE-FILTER VALIDATION. THE FIRST BLOCK COVERS THE 385-RECORD VERIFICATION SET; THE SECOND REPORTS HOW FAR THE TWO CODING PASSES CONCURRED, BEFORE ADJUDICATION.

Measure	Value
Reviewer agreement, binary (retain vs. exclude)	$\kappa = 0.79$; 95% CI $[0.72, 0.85]$; 90.4%
Reviewer agreement, three-level (incl./maybe/excl.)	$\kappa = 0.64$; 95% CI $[0.57, 0.71]$; 81.6%
Pre-filter recall vs. consensus	99.1% (114/115)
Pre-filter false-omission rate (sampled exclusions)	0.48% (1/207; 95% CI 0.01–2.66%; $\approx$9 projected over 1,860, CI $\approx$0–50)
Pre-filter over-inclusion (consensus-excluded yet retained)	64
Concordance between the two coding passes, before adjudication	
Eligibility (114 unique full-text records)	80.7% (92/114)
Primary source (48 records both passes judged includable)	83.3% (40/48)
Primary source, citation-searching arm (29 studies)	86.2% (25/29)
Differences resolved in favor of the first pass	8 (6 eligibility, 2 source)

TABLE IV
PRE-FILTER CONFUSION MATRIX OVER THE 385-RECORD VERIFICATION SET: ALL 178 MODEL-RETAINED RECORDS TOGETHER WITH A REASON-STRATIFIED 10% SAMPLE OF 207 OF THE 2,067 MODEL-EXCLUDED RECORDS. THE EXCLUSION COLUMN IS A SAMPLE RATHER THAN A POPULATION TOTAL, SO IT BOUNDS THE FALSE-OMISSION RATE BUT DOES NOT GIVE A CORPUS-WIDE PRECISION.

	Human consensus		
Pre-filter decision	Retain	Exclude	Total
Retain	114	64	178
Exclude	1	206	207
Total	115	270	385

J. Full-text eligibility outcome

The 115 retained records were assessed at full text against the locked criteria of Section IV, recording for each eligibility decision a supporting evidence quote together with the codebook fields of Section IV-H. One record was found to duplicate another, leaving 114 unique full-text records, of which 54 were included [2]–[7], [10], [15]–[20], [27]–[67] and 60 excluded. The citation-searching round contributed 29 further included studies [68]–[96], so the syntheses that follow cover 83 studies.

The dominant exclusion reason, by a wide margin, is E2 (55 of 60). On full reading, these studies use the LLM to generate test inputs, to produce specifications that a static solver then proves, to drive exploration whose only failure signal is a crash or a log, or to generate test documents for human execution. In each, the verdict is reached by a non-LLM mechanism, a differential comparator, a static verifier, or a human reviewer, rather than by an oracle the LLM supplies or determines. This criterion cannot be applied reliably from title and abstract, which accounts for the gap between the 115 apparently oracle-relevant records and the 54 in which an LLM supplies or determines the oracle. The remaining exclusions are three items that are too short or non-research (E6) and two that fail another inclusion criterion. One boundary recurred often enough to state explicitly: an LLM-generated invariant, contract, or assertion that is *executed against a running system* is treated as an oracle and included. The same artifact used only inside a *static proof*, by contrast, is treated as verification and excluded.

V. FINDINGS

Every number below is computed from the released coding sheet [22] and can be reproduced; each study's coding is listed in Table XI.

A. The landscape: who, what, and when (RQ1)

The reviewed work is recent and concentrated, most of it targeting ordinary software at the unit level: general-purpose software accounts for 54 of the 83 studies. The rest form smaller clusters: embedded or programmable logic controller (PLC) control with six, web and mobile with five each, security with three, two studies each in representational state transfer (REST) or API services, graphical user interfaces (GUIs), machine or deep learning (ML/DL) systems, and autonomous driving systems (ADSs), and the remaining two domains contributing one study each (Fig. 5). The source of authority varies with the domain. Implementation-derived oracles are almost all general-purpose unit-level software, 16 of 17. Model-parametric oracles split between the neural assertion-generation line on general-purpose software (8 of 17) and interactive or visual domains such as mobile apps, GUIs, and ADSs (7 of 17) [18], [54], with the remaining two in ML/DL and embedded code. In the interactive domains a plausible reason is that no executable specification is on offer, leaving the model's own expectations as often the only authority within reach. One study is from 2022, five are from 2023, ten from 2024, 43 from 2025, and 24 from 2026. As a result, 81% appeared in 2025 or later (Fig. 3). All included work is peer reviewed, spread across 40 venues but led by the main software-engineering ones (Table V). Conferences carry most of it (51 of 83), with journals a strong second (29) and workshops the remaining three.

The studies lean on closed-weight models and on prompting rather than training. Setting aside the 39 studies that compare several models, the GPT family is the most common single choice: 20 studies use a GPT model as their primary, a GPT-4-class model in 16 and GPT-3.5 in 4. Most use the model as it comes, with fine-tuning in 22 (Fig. 6). Closed models (35) outnumber open-weight ones (23), with 25 studies using both

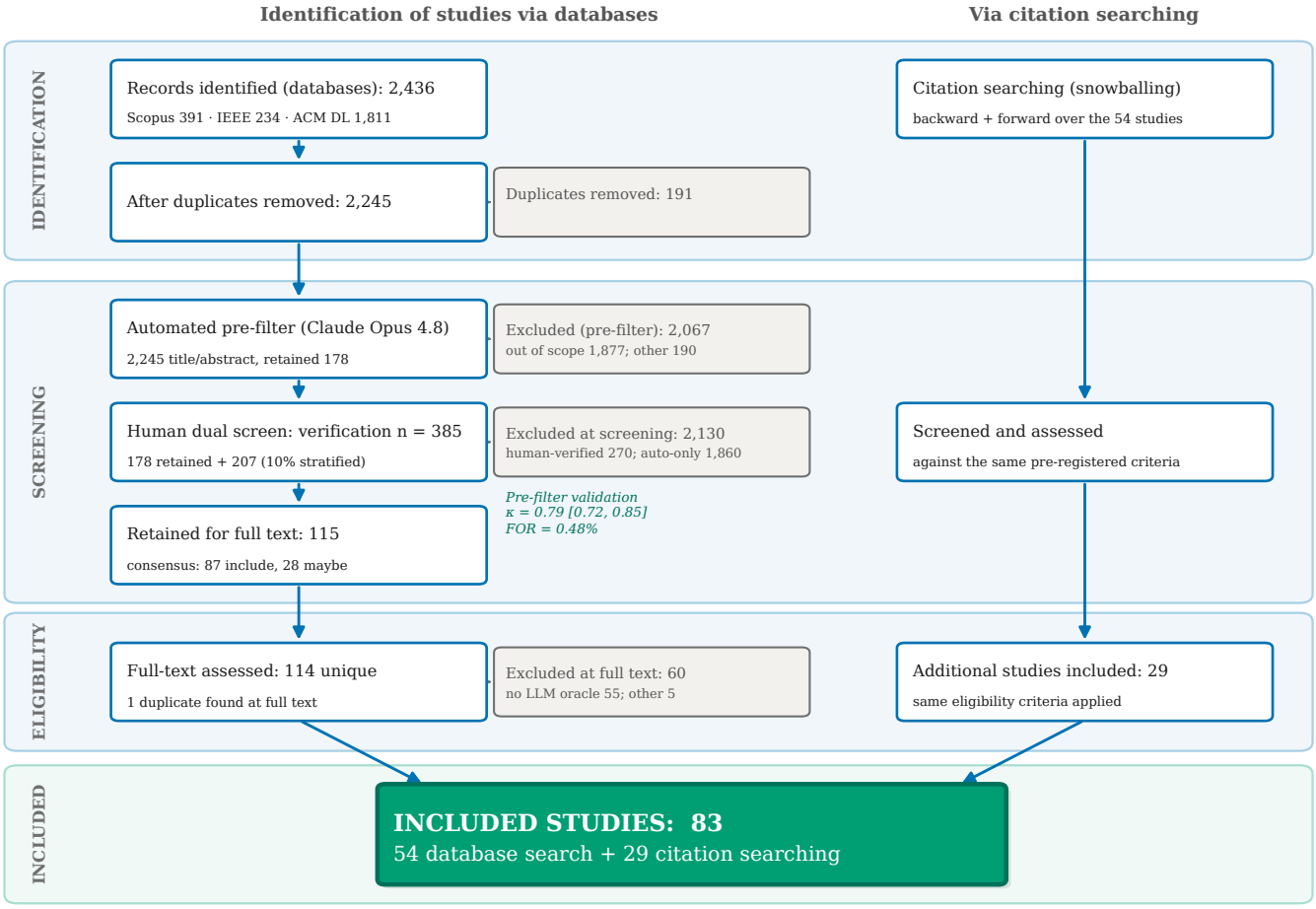


Fig. 2. PRISMA 2020 flow of records: the database-search arm (left, full detail) and the citation-searching arm (right, summarized), converging on the 83 included studies. Per-stage exclusion reasons are in the supplementary material.

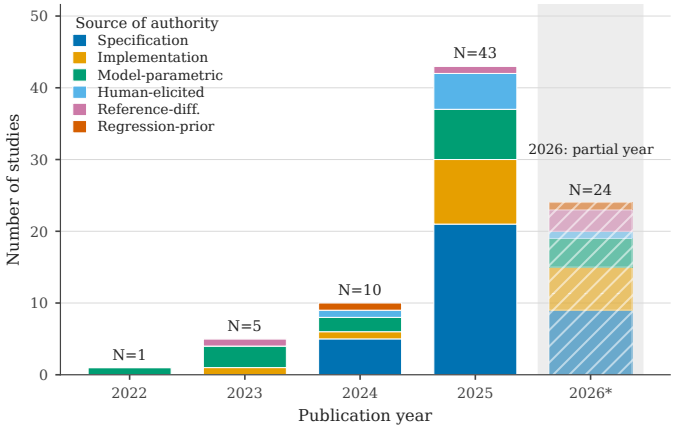


Fig. 3. Included studies by publication year, stacked by source of authority (2026 partial, hatched).

(Fig. 4(a)). Open-science practice is common: 65 of 83 release code or data (Fig. 4(b)), and most studies evaluate or validate existing ideas rather than propose new ones (Fig. 4(c)). The languages tested are mostly Java (35) and Python (18).

B. *Where the authority comes from (RQ2.a)*

The single largest group of reviewed oracles draws its authority from a written description of intended behavior (Fig. 7). Specification-derived oracles account for 35 of the 83. Within them, natural-language artifacts, such as docstrings, code comments, requirements, and API documentation, account for 23, ahead of semi-formal (7) and formal (5). Implementation-derived oracles, which read expected behavior from the code or existing tests, and model-parametric oracles, which rely on the model's own knowledge with no external reference, tie as the next largest groups (17 each). Human-elicited oracles follow (7). Reference-differential oracles (5) include a recent variant in which the consensus output of several model-generated alternative implementations, rather than an independent reference version, supplies the expected value [19], [20]. One of the seven categories, implicit-intrinsic, remains unoccupied as a *primary* source, appearing only as a secondary signal in compositional systems. Regression-from-prior-version, unoccupied in the database-search corpus, is now held by two studies in which a prior version's behavior anchors an LLM-generated or LLM-judged test [82], [93] (Section VI-C).

Recording one primary source per study is a coding decision

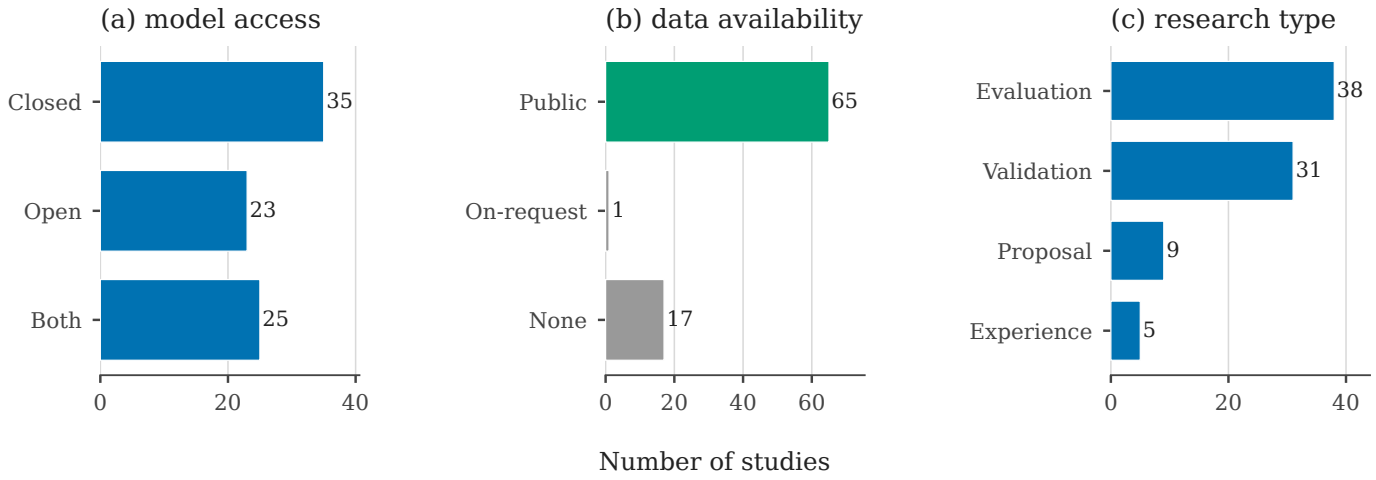


Fig. 4. Study characteristics: (a) model access, (b) data or code availability, (c) research type.

TABLE V
LEADING PUBLICATION VENUES (THOSE WITH AT LEAST TWO STUDIES; 40 VENUES IN TOTAL).

Venue	Studies
Proc. ACM Software Engineering (FSE / ISSTA)	9
Int. Conf. Software Engineering (ICSE, incl. SEIP)	7
ACM Trans. Software Engineering and Methodology (TOSEM)	6
IEEE Transactions on Software Engineering (TSE)	6
Int. Conf. Automated Software Engineering (ASE)	5
Int. Conf. Foundations of Software Engineering (FSE, incl. ESEC)	3
IEEE Conf. Software Testing, Verification and Validation (ICST)	3
Annual Meeting of the Assoc. for Computational Linguistics (ACL)	3
Information and Software Technology	3
Lecture Notes in Computer Science (various)	3
Int. Conf. AI-powered Software (AIware)	2
Int. Conf. Automation of Software Test (AST)	2
Asia-Pacific Software Engineering Conf. (APSEC)	2
Int. Symp. Software Testing and Analysis (ISSTA proceedings)	2
IEEE Int. Conf. Software Testing Workshops (ICSTW)	2
25 further venues with one study each.	

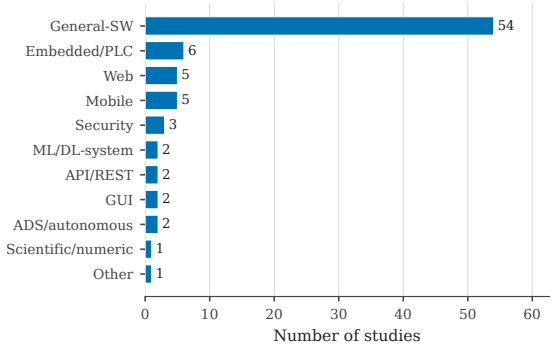


Fig. 5. Application domains of the systems under test. Abbreviated labels: General-SW, general-purpose software; ML/DL, machine or deep learning; PLC, programmable logic controller; API, application programming interface; REST, representational state transfer; GUI, graphical user interface; ADS, autonomous driving system.

rather than a property of the systems, so it is worth reading the corpus a second way, with each study's secondary source counted alongside its primary one. Sixty-one of the 83 studies record a secondary source, giving 144 source labels in all. The picture is stable where it matters most: specification-derived remains the largest group by a wide margin (42 labels), and reference-differential and regression-from-prior-version stay near the bottom (8 and 7). Two things move. Implementation-derived authority passes model-parametric (31 against 27), because code and existing tests are the most common supporting signal in compositional designs; and implicit-intrinsic, which no study uses as its primary source, appears as a secondary signal in four. The conclusions that follow rest on the primary coding and on the source-by-mechanism structure, and neither is disturbed by this second reading; what it does show is that the one empty category is a consequence of counting primary sources only, not a claim that the signal is absent from the corpus. Every secondary source is listed study by study in Table XI.

C. Specification dependence: an audit of the source coding (RQ2.a)

Specification dependence is not posed as a separate research question, because the source categories are defined by their relation to a specification; it is reported here as an audit of the RQ2.a coding. Because the sources are defined by their relation to a specification, the corpus follows the idealized split in direction, though with a wider partial fringe than the database-search corpus alone (Fig. 8). This is a consistency check rather than an empirical discovery. Specification dependence is coded as its own field, separately from the source code, so the near-perfect alignment between the two

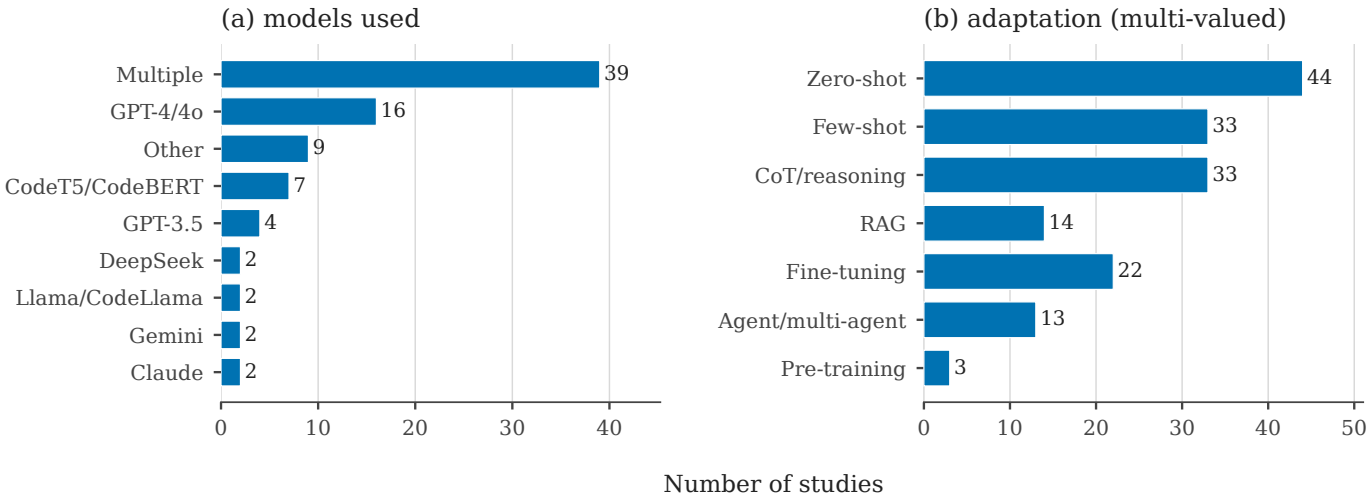


Fig. 6. LLM use: (a) primary model family per study, (b) adaptation strategies (multi-valued). CoT, chain-of-thought; RAG, retrieval-augmented generation.

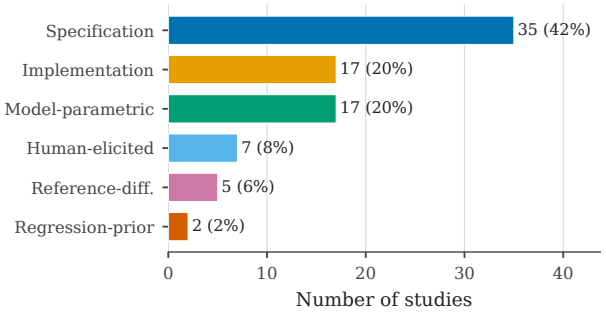


Fig. 7. Primary source of oracle authority (83 studies).

confirms that the categories behave as defined; this alignment is what is expected by construction and is reported as an audit of the taxonomy, not as a finding about the literature; the substantive result is the source distribution reported above, and the specification-independent share simply follows from it. Every implementation-derived oracle (17 of 17) renders its verdict without any external specification, as do 14 of the 17 model-parametric, six of the seven human-elicited, three of the five reference-differential, and one of the two regression-from-prior-version oracles; the remainder are partly dependent, and no non-specification source is fully specification-dependent. Specification-derived oracles, by contrast, are specification-dependent or partly so in 34 of 35 cases. The 42 fully specification-independent studies make up just over half of the corpus. All but one draw their verdict authority from the code, the model's own knowledge, an independent reference, a prior version, or a human; the exception is the specification-derived oracle that infers its own specification, discussed next.

The two crossings carry the real information. The one specification-derived oracle that needs no pre-existing specification [51] has the LLM *infer* executable metamorphic relations; the inferred relation then serves as the specification, so the spec is synthesized rather than supplied. In the database-search corpus, the one reference-differential oracle that partly depends on a specification [20] takes the consensus of LLM-synthesized reference versions as its expected output, and its coding records a secondary specification-derived source: part of the verdict's authority flows from the specification that guided the synthesis. Both show the same pattern: the boundary blurs exactly where the LLM manufactures the artifact that other designs take as given. The citation-searching extension adds six further partial crossings of the same character, among them the corpus's second partly-dependent reference-differential oracle: in each, part of the verdict's authority flows through a specification-shaped artifact that the model reads, synthesizes, or internalizes, from docstring signals inside neural assertion generators to the issue text behind a regression-catching test.

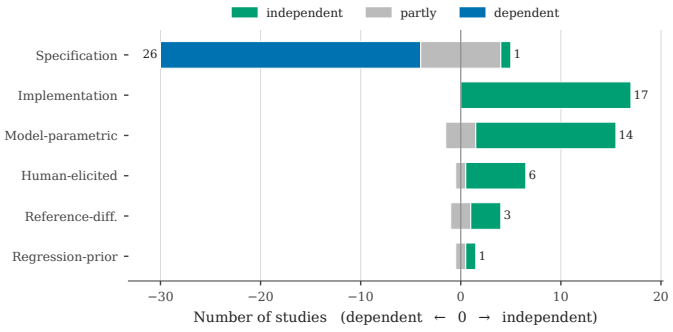


Fig. 8. Specification (in)dependence of the verdict, by primary source of authority.

D. How oracles are expressed and adjudicated (RQ2.b, RQ2.c)

Table VI reports oracle form and adjudication mechanism, both multi-valued, so column totals exceed 83. Assertions dominate the form (56) and runtime assertion execution the mechanism (56). The key pattern is the division of labor: in 65 of 83 studies the LLM generates an oracle artifact that some other mechanism then checks, in 16 it is itself the judge, and in two it validates an oracle produced by other means.

TABLE VI
ORACLE FORM, ADJUDICATION MECHANISM, AND LLM ROLE (FORM AND MECHANISM MULTI-VALUED; TOTALS EXCEED 83). PBT, PROPERTY-BASED TESTING.

Oracle form (RQ2.b)	
Assertion	56
Expected-output value	19
Classifier / judge label	15
Metamorphic relation	12
Invariant	11
Property / PBT	9
Other (exception oracle)	2
Adjudication mechanism (RQ2.c)	
Runtime assertion execution	56
Exact / structural match	26
LLM-as-judge	22
Human review	11
Statistical / distributional	8
Formal verification	6
Semantic / similarity	4
LLM role (RQ2.c)	
Generates oracle artifact (checked otherwise)	65
Is the oracle / judge	16
Validates an oracle produced by other means	2

The two axes do not line up one to one (Fig. 9). Specification-derived authority is decided by all seven mechanisms; conversely each mechanism draws on between three and six different sources. Runtime assertion execution, the most common mechanism (56 co-occurrences), serves every occupied source. LLM-as-judge (22) serves specification-derived, model-parametric, human-elicited, and regression-from-prior-version authority, and appears once as a validation step over an implementation-derived oracle [75]. The LLM's role adds a second pattern. In all 16 studies where the LLM is itself the judge, its authority is a specification, its own parametric knowledge, human-elicited criteria, or a prior version, and in none is it implementation-derived or reference-differential. With only 16 such studies this is a pattern in the corpus rather than an established constraint. For those two sources the LLM writes an artifact that some other mechanism runs. Cataloging oracles by mechanism or form alone, as prior secondary studies do, discards this structure.

E. How well the oracles work (RQ3.a)

Most studies compare against a baseline (65 of 83), but the evaluation ground is fragmented and the oracle's own correctness is unevenly checked. 64% rely on a custom benchmark (53 of 83). The most common shared benchmarks are Defects4J (16) and HumanEval (7), which limits cross-study comparison. Ground truth is most often human-provided (37) or taken from existing tests (20), with reference implementations (15) and specifications (8) behind. Oracle quality is assessed against ground-truth oracles in 52 studies and by manual inspection in 19; only 13 use mutation analysis to test whether the oracle actually catches injected faults, and 3 do not assess it at all. The dominant reported metrics are accuracy against a reference (39) and fault detection (35), followed by precision and recall (31), coverage (19), and mutation score (19). Resemblance to a known oracle is thus the dominant quality signal, rather than mutation-grounded fault detection.

F. How the oracles fail (RQ3.b)

Reported failure modes converge on a small set (Fig. 10). Hallucination is the model asserting behavior the system does not exhibit. It is the hazard studies engage with most, surfacing in 60 of 83 studies. A closer reading separates three overlapping ways studies engage with it (Table VIII): 58 name it as a risk, 44 report observing it empirically in their own results, and 46 apply a concrete mitigation. Among the other modes, the leaders are weak or missing assertions (38), where the generated oracle is too loose to detect the faults it should, and prompt sensitivity (29); then cost or latency (26), flakiness or nondeterminism (20), and data contamination (20). Over-assertion, an oracle so strict that it rejects correct behavior, appears in only 11 studies, against 38 for weak or missing assertions, making weak oracles the most frequently reported correctness risk. But how often a failure is reported may not match how often it happens. The 46 mitigations are led by execution-based filtering, compiling and running a candidate oracle and discarding what fails to compile or rejects known-correct behavior [16], [58]; self-consistency or multi-sample voting [19], [20]; and human review (11 studies, Table VI).

G. Study quality appraisal

To characterize the rigor of the corpus without excluding any study, each of the 83 included studies was scored on a lightweight nine-item checklist derived from the coded fields (Table IX). The corpus is uniformly strong on the items the taxonomy relies on: every study defines its oracle, identifies an authority source, and discusses its own limitations. Almost all also report a benchmark (75 of 83) and a ground-truth source (80). It is weakest exactly where the evaluation findings (RQ3.a) already pointed: fault detection is evaluated in 35 studies and mutation analysis used, as a metric or as an oracle check, in only 20. We report the nine items individually rather than as a summed score: they mix reporting, artifact-availability, and fault-detection constructs that do not form a single validated quality index. Read item by item, they describe a literature that is mature in reporting but light on fault-grounded evaluation, so findings that lean chiefly on the fault-grounded items (RQ3.a) rest on the weaker part of the corpus.

VI. DISCUSSION

A. What the findings mean

Three results stand out, together showing a field that has changed what an oracle rests on.

First, the authority behind LLM-based oracles often does not come from a specification at all. That makes them useful on everyday software, which rarely has a written specification. It also makes them risky: an oracle resting only on the model's own knowledge, with no external anchor at all, can be wrong with no signal that it is.

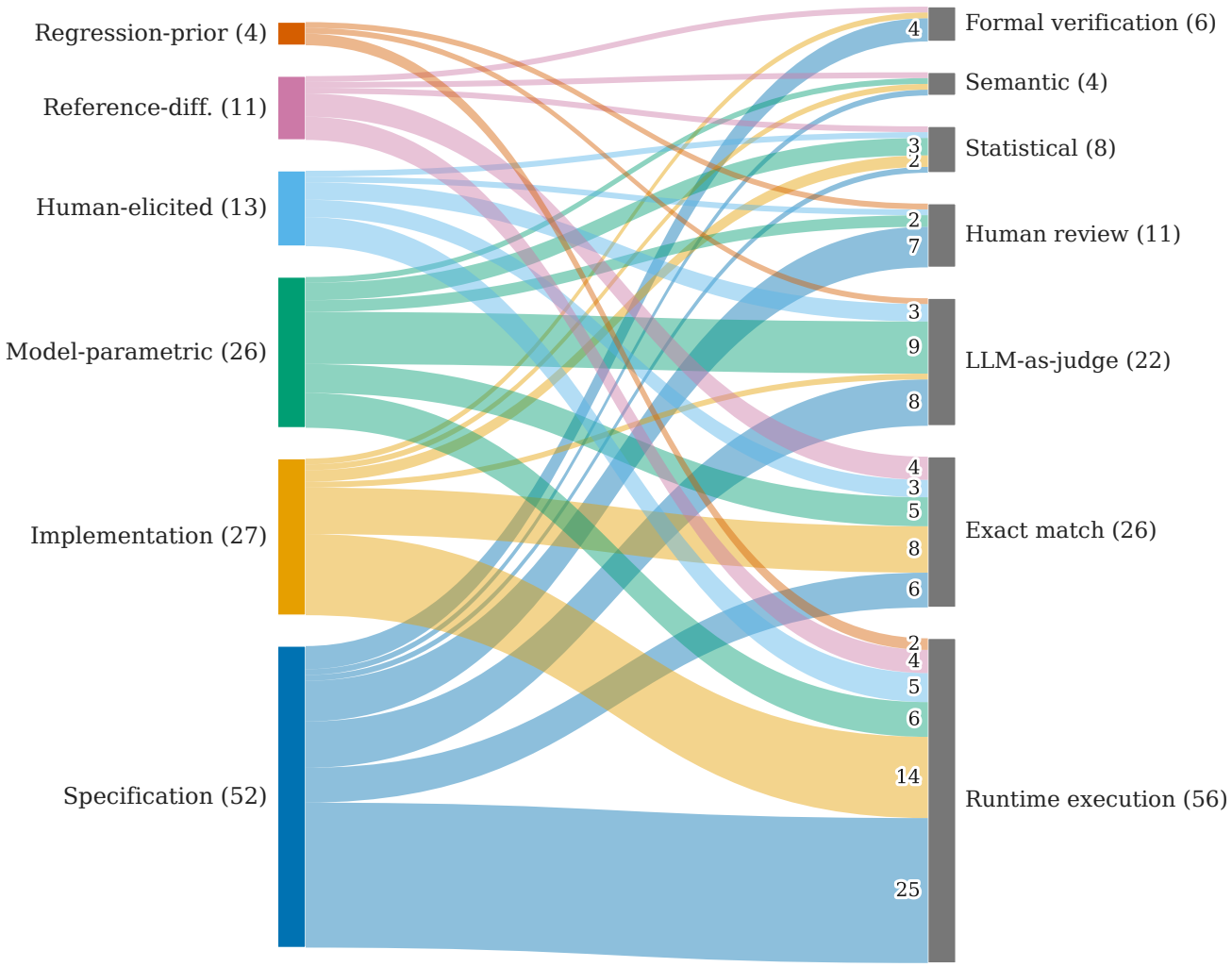


Fig. 9. Source of authority (left) to the mechanism that adjudicates it (right). Parenthetical numbers are node totals. Each ribbon's number is that source's contribution. All are study co-occurrences (one per mechanism a study uses), so the source totals exceed the primary-source counts of Fig. 7. Unlabeled ribbons are single studies. Each study contributes its single primary source paired with each mechanism it uses, so no source combinations are introduced; a numerical version is given in Table VII.

TABLE VII
NUMERICAL SOURCE-BY-MECHANISM CO-OCCURRENCE MATRIX BEHIND FIG. 9: ONE COUNT PER (SOURCE, MECHANISM) PAIR A STUDY REALIZES, OVER THE SEVEN PRINCIPAL MECHANISMS. ROW AND COLUMN TOTALS ARE THE NODE TOTALS OF THE FIGURE (133 CO-OCCURRENCES ACROSS THE 83 STUDIES). THREE STUDIES ADDITIONALLY RECORD AN *other* MECHANISM (MUTATION TESTING, A DIFFERENTIAL CROSS-CHECK, AND A RULE-BASED MATCHER), LISTED IN THE APPENDIX BUT OUTSIDE THESE SEVEN. SOURCES ARE SINGLE-VALUED PER STUDY, SO NO SOURCE COMBINATIONS ARISE.

Source	Runtime exec	Exact match	LLM-judge	Human review	Statistical	Semantic	Formal verif	Total
Specification-derived	25	6	8	7	1	1	4	52
Implementation-derived	14	8	1	0	2	1	1	27
Model-parametric	6	5	9	2	3	1	0	26
Human-elicited	5	3	3	1	1	0	0	13
Reference-differential	4	4	0	0	1	1	1	11
Regression-from-prior-version	2	0	1	1	0	0	0	4
Total	56	26	22	11	8	4	6	133

Second, the source of authority and the way pass or fail is computed are separate choices (Fig. 9). A label such as "LLM-as-judge" therefore says nothing about how far to trust the verdict.

Third, the most frequently reported risks concern oracle strength: the most common reported *correctness* failure is an oracle too weak to catch the bug it should. This compounds the hazard studies engage with most, the model's tendency to hallucinate behavior the system never shows.

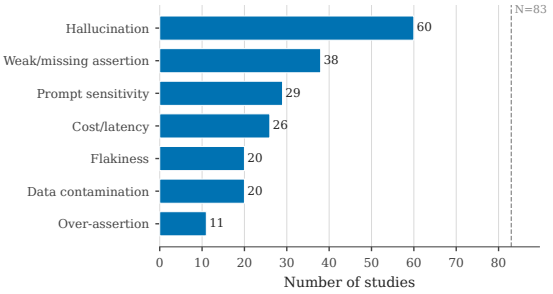


Fig. 10. Reported failure modes (multi-valued; dashed line marks the 83 studies). One-off study-specific modes are omitted.

TABLE VIII
HOW STUDIES ENGAGE WITH HALLUCINATION, THE MOST COMMON FAILURE MODE (CATEGORIES OVERLAP).

Engagement with hallucination	Studies
Named as a risk or threat	58
Empirically observed in own results	44
Mitigation applied	46
60 of 83 studies engage in at least one of these ways.	

TABLE IX
LIGHTWEIGHT QUALITY APPRAISAL OF THE 83 INCLUDED STUDIES (NON-EXCLUSIONARY; EACH ITEM SCORED 0/1).

Quality item	Studies
Oracle explicitly defined	83 (100%)
Authority source identifiable	83 (100%)
Threats / limitations discussed	83 (100%)
Ground truth available	80 (96%)
Dataset / benchmark described	75 (90%)
Code or data released	65 (78%)
Baseline comparison reported	65 (78%)
Fault detection evaluated	35 (42%)
Mutation analysis used (metric or oracle check)	20 (24%)
Items reported individually; not summed into a composite score.	

B. Choosing a source of authority in practice

The taxonomy also reads as a practical guide: the right source depends on what the project already has.

If the code has documentation, such as docstrings or requirements, specification-derived oracles are the best-supported path. The reviewed work most often turns natural-language documentation into assertions that run against the program [2], [4], [16].

If there is no specification but a trusted second implementation exists, a reference-differential oracle is natural: run both and compare. A newer variant has the LLM write several implementations and treats their agreement as the expected answer, removing the need for an existing reference [19], [20].

If the program is interactive or visual, such as a mobile app or a GUI, the field uses model-parametric oracles. The LLM judges the output directly from its own knowledge, because no executable specification is available [5], [6]. This is the least anchored option, so it pairs best with human review or independent cross-checks.

If existing tests already encode expected behavior, implementation-derived oracles can extend them. They inherit the limits of those tests, however, and cannot catch a bug already in the current code.

In every case the source should be stated plainly.

How the model obtains that authority does not change which source it is. A study that uses retrieval-augmented generation (RAG) or an agentic tool call to fetch a specification is still specification-derived; one that fetches or executes existing code is still implementation-derived. Twenty-four of the 83 studies use RAG or agentic (multi-agent) adaptation (14 and 13 respectively, with 3 studies using both). They span six sources (12 specification-derived, 5 implementation-derived, 3 model-parametric, 2 human-elicited, 1 reference-differential, 1 regression-from-prior-version), indicating that adaptation strategy and source of authority are separate, cross-cutting choices.

C. Gaps and a research agenda (RQ3.c)

Reading the sparse and empty regions of the source-by-mechanism space (Fig. 9) and the source totals (Fig. 7) yields a concrete agenda. The gaps below are an interpretive reading of where LLM-based oracles have not yet been applied, not an observed empirical result; several are shaped by our review scope (Section VIII) rather than by the state of practice.

Near-empty source categories. One of the seven sources remains unoccupied, and one nearly so. No included study rests on a purely implicit-intrinsic oracle (crashes or well-formedness) as the LLM's contribution: that failure signal is supplied without an LLM, so such designs fall to E2, and the category surfaces only as a *secondary*, compositional signal (in four studies). This cell is therefore empty partly by construction of our scope. Regression-from-prior-version authority, unoccupied in the database-search corpus, is now held by two studies in which a prior version's behavior anchors an LLM-generated or LLM-judged catching test [82], [93]; with two studies it is the sparsest occupied source. Whether an LLM can add value in the implicit-intrinsic cell, where it is currently bypassed, this corpus cannot settle.

Empty source-by-mechanism cells. LLM-as-judge is never applied to reference-differential authority, and touches implementation-derived authority only once, as a validation step over an inferred contract [75]: when the authority is the code or an independent reference, the included studies otherwise execute or match rather than asking the model to judge. Whether an LLM judge briefed with the implementation or the reference could outperform exact matching, particularly for outputs that are semantically equivalent but textually different, is almost untested. Human-elicited authority, sparse in the database-search corpus, now covers seven studies spanning five mechanisms.

Evaluation gaps. The reliance on custom benchmarks (64%) and the thin use of mutation analysis to validate oracle fault detection (13 of 83) together mean the field lacks a shared, fault-grounded yardstick for oracle quality. A standard oracle-evaluation benchmark scored by mutation killing, rather than by textual similarity to a reference oracle, would be valuable.

Trust gaps. Hallucination is the hazard studies engage with most (60 of 83), but it is named far more often than measured: 58 studies flag it as a risk while 44 report observing it. The 46 mitigations, in turn, are varied and rarely evaluated for residual risk. The weak-assertion failure mode (38) shows that generated oracles frequently under-constrain behavior in a way that similarity metrics do not detect. Measuring oracle *strength*, beyond oracle-generation accuracy, is the open problem these findings point to.

VII. Related Reviews

Fourteen secondary studies overlap this review's scope [8], [9], [97]–[108]. The full per-study analysis is given in Table X. None organizes the oracle literature by the source of verdict authority. Where they address the oracle at all, thirteen treat it by output form or by LLM technique, and one partitions it by mechanism [101]. The only oracle-dedicated secondary study is a non-systematic roadmap [98]. None reports an inter-reviewer agreement coefficient, even among the broad reviews of LLMs for software engineering [8], [9]. Nor do they separate the three questions this review keeps apart: what shape the oracle takes, how pass or fail is computed, and where its authority comes from. Among the surveys we identified, none is simultaneously oracle-focused, systematically searched, organized by source of authority, and reliability-reported; that is the position this review occupies.

TABLE X
This review against the 14 related secondary studies (counts over the 14).

Property	Prior reviews (of 14)	This review
Oracle is the primary unit of analysis	1†	yes
Systematic search with a reported funnel	7	yes
Organized by source of verdict authority	0	yes
Inter-reviewer agreement (κ) reported	0	yes

†The one oracle-focused secondary study is a non-systematic roadmap (no search funnel). One further review partitions oracles by mechanism, counted as *not* organized by source of authority.

VIII. Threats to Validity

A. Search and screening validity

The recall-tuned query admitted many off-topic records. These were removed by criterion-based screening (Table III).

The pre-filter's exclusions carry the only residual risk: the model-excluded records not individually re-screened. The audited sample puts the false-omission rate at 0.48% (1/207, Clopper–Pearson exact 95% CI 0.01–2.66%), projecting to roughly nine (95% CI ≈0–50) missed records across the 1,860 unaudited exclusions. Because the CI around a single observed omission is wide, this residual risk is disclosed here, with the sampled decisions released for audit. It is unlikely to change the main taxonomy-level findings, and its consequence can be bounded. A missed record would still have had to clear full-text eligibility, which 54 of the 114 unique full-text records did, so the nine projected omissions correspond to roughly four further included studies. Distributed like the rest of the corpus, they would leave every source ranking unchanged. The finding most sensitive to them is the narrow majority that reaches a verdict without a specification (42 of 83): a worst case in which all four additions were specification-derived would turn that majority into a narrow minority, though the substantive point, that a large share of these oracles need no specification at all, holds either way.

B. Scope validity

The search covered Scopus, IEEE Xplore, and the ACM Digital Library; Web of Science, Springer Link, and dedicated preprint servers were not queried, and preprint-only work is therefore under-represented (criterion E5). The citation-searching round partially mitigates this boundary, reaching work indexed outside the three databases through the citation graph of the included studies.

LLM-based assertion generation for hardware (criterion E3) is a sizable adjacent literature that we deliberately place out of scope. It is a natural target for a companion review.

C. Classification validity

The source-of-authority code is the review's central interpretive judgment and is, like any qualitative coding, subject to reasonable disagreement at category boundaries. To make it auditable, every included and excluded record is released [22] with its eligibility decision, the criterion applied, the primary and secondary source codes, and an anchoring quotation. The load-bearing boundary rules, notably runtime execution versus static proof, the LLM as oracle versus as test-input generator, and a supplied specification versus a bare task prompt (Section III), are stated explicitly so that any classification can be re-traced and contested. The source-of-authority coding (Section IV-H) is interpretive, so a misclassification could pass undetected; the anchoring quotations in the released sheet are the audit trail for exactly this risk. As a sensitivity check, reassigning the two contested primary-source codings [27], [28] to their alternative readings changes no aggregate count by more than one study and no conclusion of the review.

IX. Conclusion

Test oracles decide whether software behaves correctly, and LLMs have made it possible to generate them or to use a model as one. This review organized that new body of work by a question the literature usually skips: where does the verdict's authority come from? Across 83 studies, the single most common authority is a written description, but just over half need no specification at all. The source categories track this split by construction, and the corpus confirms it, crossed only by a small partial fringe where the model itself supplies or reads the specification-shaped artifact other designs are handed. The source of authority cross-cuts how pass or fail is computed, so describing an oracle only by its mechanism leaves out what matters most. The most frequently reported weaknesses are oracles too weak to catch bugs, and verdicts the model can hallucinate. Its evaluation usually checks resemblance to a known oracle, and rarely whether the oracle

catches faults. The source-of-authority taxonomy reads this literature on the grounds that set how far a verdict can be trusted. For practitioners, the first question about any LLM oracle should be where its authority comes from, since that, more than the mechanism, is what a pass or fail can ultimately be defended on. These findings carry the residual uncertainties disclosed in Section VIII: a small number of records the automated pre-filter may have missed, databases and preprint venues not searched, and the interpretive nature of qualitative classification.

APPENDIX A
PER-STUDY CATEGORIZATION

Table XI lists every included study with its primary and, where compositional, secondary source of authority, its specification-dependence label, all coded oracle forms and adjudication mechanisms, and application domain, so the multi-valued totals of Tables VI and VII can be reconstructed row by row. It lets any aggregate count in the synthesis be traced back to the studies behind it. The complete coding sheet, with its coded fields and the anchoring quotations, is provided as supplementary material.

TABLE XI: The 83 included studies and their complete coding: primary source, secondary source where a system is compositional, specification dependence (Sp), and all applicable oracle forms and adjudication mechanisms, so every multi-valued aggregate in the synthesis (Tables VI and VII) can be reconstructed from this table. Column abbreviations are defined in the note beneath the table.

ID	Yr	Source	2nd src	Sp	Form(s)	Mechanism(s)	Domain
0001 [29]	2025	Spec	–	D	Assert; Exp-out	Exact	web
0009 [30]	2025	Spec	Human	D	Assert; Property	Human-rev; RT-exec	general-SW
0014 [31]	2025	Spec	Human	D	Metam	LLM-judge; Human-rev	scientific/numeric
0017 [32]	2025	Human	Ref-diff	I	Judge	LLM-judge	general-SW
0018 [33]	2026	Ref-diff	Model	I	Exp-out; Metam	Exact; Statist	ML/DL-system
0041 [18]	2024	Model	Human	I	Judge	LLM-judge	mobile
0043 [34]	2025	Impl	Human	I	Assert; Exp-out	Exact; Statist	general-SW
0048 [35]	2026	Impl	–	I	Invar	RT-exec	general-SW
0072 [36]	2026	Model	Regression	I	Assert	Exact; Statist	ML/DL-system
0083 [37]	2026	Spec	Impl	P	Exp-out	Exact	embedded/PLC
0087 [38]	2025	Spec	Impl	D	Invar	LLM-judge	API/REST
0091 [39]	2024	Spec	Model	P	Assert; Exp-out	LLM-judge; RT-exec	general-SW
0098 [3]	2025	Impl	Regression	I	Assert	Exact; RT-exec	general-SW
0103 [40]	2025	Human	Ref-diff	I	Judge	LLM-judge; Statist	general-SW
0129 [41]	2025	Impl	Model	I	Assert	Exact	general-SW
0132 [42]	2025	Impl	–	I	Assert	RT-exec	general-SW
0137 [43]	2025	Spec	–	D	Assert; Property	RT-exec	API/REST
0161 [44]	2024	Impl	Human	I	Assert	Exact; RT-exec; Semantic	general-SW
0177 [45]	2026	Spec	Impl	D	Assert; Property	RT-exec	embedded/PLC
0183 [46]	2024	Spec	–	D	Assert	RT-exec	general-SW
0184 [27]	2025	Model	Impl	I	Assert	Human-rev; RT-exec	general-SW
0211 [47]	2024	Spec	–	D	Metam	RT-exec	web
0213 [48]	2025	Spec	Impl	D	Assert; Metam	LLM-judge; Statist	general-SW
0223 [49]	2025	Spec	Human	D	Assert	Exact; RT-exec	general-SW
0231 [50]	2025	Model	Implicit	I	Metam	Exact	embedded/PLC
0248 [10]	2025	Spec	Model	P	Metam	Human-rev; RT-exec	general-SW
0259 [2]	2025	Spec	Model	P	Assert; Other	Exact; RT-exec	general-SW
0261 [51]	2026	Spec	Impl	I	Metam	RT-exec	general-SW
0279 [52]	2025	Spec	Model	D	Assert; Property	RT-exec	general-SW
0287 [17]	2025	Model	Human	I	Judge	LLM-judge	GUI
0304 [53]	2025	Spec	–	D	Assert; Exp-out	RT-exec	general-SW
0316 [54]	2023	Model	Human	I	Metam	Human-rev	ADS/autonomous
0320 [55]	2025	Spec	Impl	D	Assert	RT-exec	mobile
0323 [56]	2025	Spec	Implicit	P	Metam	Human-rev; RT-exec	ADS/autonomous
0339 [15]	2025	Spec	–	D	Assert; Exp-out	RT-exec	general-SW
0371 [57]	2025	Impl	Regression	I	Assert	Exact; RT-exec	general-SW
0372 [5]	2025	Model	Human	I	Judge	LLM-judge	mobile
0388 [58]	2025	Impl	Model	I	Assert	RT-exec	general-SW
0390 [59]	2026	Spec	–	D	Judge	LLM-judge	web
0406 [60]	2025	Impl	Spec	I	Assert; Invar; Property	Formal; RT-exec	security
0408 [61]	2025	Spec	Impl	D	Assert; Invar; Property	Human-rev; RT-exec	security
0413 [62]	2025	Impl	–	I	Assert	Exact	general-SW
0417 [6]	2025	Model	Human	I	Judge	LLM-judge	GUI
0464 [63]	2026	Model	Implicit	I	Judge; Metam	LLM-judge; Semantic	mobile
0698 [16]	2024	Spec	–	D	Assert	RT-exec	general-SW
0699 [19]	2023	Ref-diff	Model	I	Exp-out	Exact; RT-exec	general-SW
1131 [4]	2025	Spec	Impl	P	Assert; Exp-out	Exact; RT-exec	general-SW
1154 [64]	2025	Impl	Regression	I	Assert; Exp-out	Exact; RT-exec	general-SW
1311 [65]	2025	Spec	–	D	Exp-out; Property	Exact; RT-exec	embedded/PLC
1725 [7]	2025	Spec	Human	D	Assert; Judge	LLM-judge; RT-exec	web
1846 [66]	2026	Spec	Impl	D	Assert; Exp-out	LLM-judge; RT-exec	general-SW
1850 [28]	2026	Ref-diff	–	I	Metam	Exact; Formal; RT-exec	other
1985 [20]	2026	Ref-diff	Spec	P	Assert; Exp-out	RT-exec; Semantic	general-SW
2138 [67]	2026	Spec	Ref-diff	P	Judge	LLM-judge; Semantic	general-SW
2246 [68]	2022	Model	Human	I	Assert; Exp-out	LLM-judge; RT-exec	general-SW
2247 [69]	2023	Model	Spec	P	Assert; Judge	Exact; RT-exec	general-SW
2248 [70]	2023	Model	Human	P	Assert; Exp-out; Other	RT-exec	general-SW
2249 [71]	2026	Impl	–	I	Assert; Metam	Exact; RT-exec; Statist	general-SW
2250 [72]	2026	Impl	–	I	Assert	Other; RT-exec	general-SW
2251 [73]	2024	Human	–	I	Assert	Exact; RT-exec	general-SW
2252 [74]	2026	Human	Model	I	Assert; Exp-out	LLM-judge; Exact; Human-rev; RT-exec	embedded/PLC
2253 [75]	2026	Impl	Model	I	Assert; Invar	LLM-judge; RT-exec	general-SW
2254 [76]	2026	Spec	Implicit	D	Assert; Invar	RT-exec	web
2255 [77]	2025	Spec	Impl	P	Assert; Property	RT-exec	general-SW
2256 [78]	2025	Model	Human	I	Judge	LLM-judge	mobile
2257 [79]	2024	Spec	Impl	D	Assert; Invar	Formal	general-SW
2258 [80]	2026	Model	Spec	P	Assert; Exp-out	LLM-judge; Exact; RT-exec; Statist	general-SW
2259 [81]	2026	Spec	–	D	Assert	Other; RT-exec	general-SW

continued on next page

Table XI continued

ID	Yr	Source	2nd src	Sp	Form(s)	Mechanism(s)	Domain
2260 [82]	2026	Regression	Human	P	Assert; Judge	LLM-judge; Human-rev; Other; RT-exec	general-SW
2261 [83]	2026	Impl	Human	I	Assert	RT-exec	general-SW
2262 [84]	2026	Impl	–	I	Assert	RT-exec	general-SW
2263 [85]	2025	Model	Impl	I	Assert; Judge	LLM-judge	general-SW
2264 [86]	2025	Human	–	I	Assert	RT-exec	general-SW
2265 [87]	2025	Human	Regression	I	Assert	Exact; RT-exec	general-SW
2266 [88]	2026	Spec	–	D	Assert; Invar	Formal; Human-rev	embedded/PLC
2267 [89]	2025	Spec	Human	D	Invar	Formal	general-SW
2268 [90]	2025	Human	Spec	P	Assert	RT-exec	general-SW
2269 [91]	2025	Spec	–	D	Assert; Invar	Formal; Human-rev; RT-exec	security
2270 [92]	2026	Model	Impl	I	Assert; Judge	Statist	general-SW
2271 [93]	2024	Regression	–	I	Assert	RT-exec	general-SW
2272 [94]	2025	Ref-diff	Spec	P	Exp-out	Exact; RT-exec	general-SW
2273 [95]	2024	Model	Human	I	Assert; Exp-out	Exact; RT-exec	general-SW
2274 [96]	2023	Impl	Spec	I	Invar; Property	RT-exec	general-SW

Abbreviations. **Source of authority:** Spec, specification-derived; Impl, implementation-derived; Model, model-parametric; Ref-diff, reference-differential; Human, human-elicited; Regression, regression-from-prior-version; Implicit, implicit-intrinsic (a dash denotes no secondary source). **Specification dependence (Sp):** D, dependent; P, partly; I, independent. **Oracle form:** Assert, assertion; Exp-out, expected-output value; Metam, metamorphic relation; Invar, invariant; Property, property-based testing (PBT); Judge, classifier/judge label; Other, exception oracle. **Adjudication mechanism:** RT-exec, runtime assertion execution; Exact, exact/structural match; LLM-judge, large language model as judge; Human-rev, human review; Statist, statistical/distributional; Semantic, semantic/similarity; Formal, formal verification. **Domain:** general-SW, general-purpose software; ML/DL, machine/deep learning; PLC, programmable logic controller; API/REST, application programming interface/representational state transfer; ADS, autonomous driving system; GUI, graphical user interface.

Data Availability

The replication package for this review is openly available on Zenodo [22] under a CC BY 4.0 license. Its concept DOI, 10.5281/zenodo.21194940, always resolves to the current version. It contains the review protocol and extraction codebook, the frozen search query and screening log, the complete de-duplicated screening funnel with per-record decisions and reasons, the independent dual-screening decisions, the Stage-1 screening prompt, the per-study coding sheet (with an anchoring quotation for every study), both coding passes for each arm together with the per-record adjudication decisions, and the PRISMA 2020 flow diagram and checklist. Publisher PDFs, raw database exports, and record abstracts are not redistributed for copyright reasons. The released per-database exports contain bibliographic metadata only.

Acknowledgment

The authors thank Noor Fatima (National University of Sciences and Technology, Pakistan) for her help with the final review and corrections of the manuscript. An LLM (Anthropic Claude) was used as the recall-oriented screening pre-filter described in the Methodology, disclosed under PRISMA 2020 item 8. Every retained and sampled screening decision was verified by independent human dual screening.

References

[1] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015, doi: 10.1109/TSE.2014.2372785.

[2] S. B. Hossain and M. B. Dwyer, "TOGLL: Correct and Strong Test Oracle Generation with LLMs," *Proc. International Conference on Software Engineering*, 2025, doi: 10.1109/icse55347.2025.00098.

[3] S. Primbs, B. Fein, and G. Fraser, "AsserT5: Test Assertion Generation Using a Fine-Tuned Code Language Model," *Proc. IEEE/ACM International Conference on Automation of Software Test, AST*, 2025, doi: 10.1109/ast66626.2025.00008.

[4] S. B. Hossain, R. Taylor, and M. Dwyer, "Doc2OracLL: Investigating the Impact of Documentation on LLM-Based Test Oracle Generation," *Proc. ACM Softw. Eng.*, 2025, doi: 10.1145/3729354.

[5] Z. Liu *et al.*, "Seeing is Believing: Vision-Driven Non-Crash Functional Bug Detection for Mobile Apps," *IEEE Transactions on Software Engineering*, vol. 51, no. 12, pp. 3452–3466, 2025, doi: 10.1109/tse.2025.3614469.

[6] Y. Hu *et al.*, "KuiTest: Leveraging Knowledge in the Wild as GUI Testing Oracle for Mobile Apps," *Proc. IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, doi: 10.1109/icse-seip66354.2025.00009.

[7] A. Chevrot, A. Vernotte, J. Falleri, X. Blanc, B. Legeard, and A. Cretin, "Are Autonomous Web Agents Good Testers?," *Proc. ACM Softw. Eng.*, 2025, doi: 10.1145/3728879.

[8] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software Testing with Large Language Models: Survey, Landscape, and Vision," *IEEE Transactions on Software Engineering*, 2024, doi: 10.1109/tse.2024.3368208.

[9] X. Hou *et al.*, "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Transactions on Software Engineering and Methodology*, 2024, doi: 10.1145/3695988.

[10] J. Zhang, C.-A. Sun, H. Liu, and S. Dong, "Can Large Language Models Discover Metamorphic Relations? A Large-Scale Empirical Study," *Proc. IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER*, 2025, doi: 10.1109/saner64311.2025.00011.

[11] M. J. Page *et al.*, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, no. n71, 2021, doi: 10.1136/bmj.n71.

[12] M. Taromirad and P. Runeson, "Assertions in software testing: Survey, landscape, and trends," *International Journal on Software Tools for Technology Transfer*, vol. 27, no. 1, pp. 117–135, 2025, doi: 10.1007/s10009-025-00794-1.

[13] T. Y. Chen *et al.*, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–27, 2018, doi: 10.1145/3143561.

[14] M. A. Gulzar, Y. Zhu, and X. Han, "Perception and practices of differential testing," in *Proc. IEEE/ACM 41st Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 71–80, doi: 10.1109/ICSE-SEIP.2019.00016.

[15] S. M. Khandaker, F. Kifetew, D. Prandi, and A. Susi, "AugmenTest: Enhancing Tests with LLM-Driven Oracles," *Proc. IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2025, doi: 10.1109/icst62969.2025.10988926.

[16] M. Endres, S. Fakhoury, S. Chakraborty, and S. K. Lahiri, "Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions?," *Proc. ACM Softw. Eng.*, 2024, doi: 10.1145/3660791.

[17] T. Rosenbach, D. Heidrich, and A. Weinert, "Automated Testing of the GUI of a Real-Life Engineering Software using Large Language Models," *Proc. IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2025, doi: 10.1109/icstw64639.2025.10962502.

[18] B. Ju *et al.*, "A Study of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps," *Proc. Asia-Pacific Software Engineering Conference, APSEC*, 2024, doi: 10.1109/apsec65559.2024.00017.

[19] T.-O. Li *et al.*, "Nuances Are the Key: Unlocking ChatGPT to Find Failure-Inducing Tests with Differential Prompting," *Proc. 38th IEEE/ACM International Conference on Automated Software Engineering*, 2023, doi: 10.1109/ase56229.2023.00089.

[20] T. Li *et al.*, "Less Is More: Failing Test Generation with Large Language Models," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3793675.

[21] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University, Joint Tech. Rep. EBSE-2007-01, 2007.

[22] A. H. Mughal and M. Bilal, "LLM-Based Test Oracles: Source-of-Authority Taxonomy - Replication Package," *Zenodo*, 2026, doi: 10.5281/zenodo.21194940.

[23] Anthropic, "Claude Opus 4.8 system card," Anthropic, San Francisco, CA, USA, May 2026. [Online]. Available: https://www.anthropic.com/claude-opus-4-8-system-card.

[24] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.

[25] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

[26] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi: 10.1093/biomet/26.4.404.

[27] A. Bodicoat, G. Jahangirova, and V. Terragni, "Understanding LLM-Driven Test Oracle Generation," *Proc. 2nd IEEE/ACM International Conference on AI-powered Software, AIware*, 2025, doi: 10.1109/aiware69974.2025.00011.

[28] Q. Mang, R. He, S. Zhong, X. Liu, H. Zhang, and A. Cheung, "Automated Discovery of Test Oracles for Database Management Systems Using LLMs," *Proc. ACM Manag. Data*, 2026, doi: 10.1145/3802017.

[29] M. Ferreira, L. Viegas, J. P. Faria, and B. Lima, "Acceptance Test Generation with Large Language Models: An Industrial Case Study," *Proc. IEEE/ACM International Conference on Automation of Software Test, AST*, 2025, doi: 10.1109/ast66626.2025.00007.

[30] D. B. Bose, "From Prompts to Properties: Rethinking LLM Code Generation with Property-Based Testing," *Proc. ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 2025, doi: 10.1145/3696630.3728702.

[31] J. Huang, M. Li, X. Yang, J. Liu, and S. Yan, "SVPrompt-MR: An LLM-Based Metamorphic Relation Identification Method with Self-Verification Mechanism," *Proc. Asia-Pacific Software Engineering Conference, APSEC*, 2025, doi: 10.1109/apsec66846.2025.00055.

[32] A. K. Vajjala *et al.*, "Using Large Language Models to Support the Workflow of Differential Testing," *Proc. ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 2025, doi: 10.1145/3696630.3728559.

[33] M. Li, D. Li, J. Liu, J. Cao, Y. Tian, and S.-C. Cheung, "Enhancing Differential Testing with LLMs for Testing Deep Learning Libraries," *ACM Transactions on Software Engineering and Methodology*, 2026, doi: 10.1145/3735637.

[34] D. Molinelli, L. D. Grazia, A. Martin-Lopez, M. D. Ernst, and M. Pezze, "Do LLMs Generate Useful Test Oracles? An Empirical Study with an Unbiased Dataset," *Proc. 40th IEEE/ACM International Conference on Automated Software Engineering, ASE*, 2025, doi: 10.1109/ase63991.2025.00031.

[35] C. Sun *et al.*, "ClassInvGen: Class Invariant Synthesis Using Large Language Models," *Lecture Notes in Computer Science*, 2026, doi: 10.1007/978-3-031-99991-8_4.

[36] G. Zeng, J. Luo, Y. Qiao, S. Teng, and A. Liu, "SpaceMutation : An LLM-assisted mutation testing framework for DNNs in distributed LEO satellites," *Expert Systems with Applications*, vol. 323, art. 132431, 2026, doi: 10.1016/j.eswa.2026.132431.

[37] B. Metekia, L. Pietrac, and K. Kouiss, "MBSE-Driven PLC Code Generation and AI-Augmented Test Scenario Validation," *Proc. International Conference on Automation, Robotics and Applications, ICARA*, 2026, doi: 10.1109/icara69401.2026.11480393.

[38] S. Fischer and C. Klammer, "Automating Invariant Filtering: Leveraging LLMs to Streamline Test Oracle Generation," *Lecture Notes in Business Information Processing*, 2025, doi: 10.1007/978-3-031-89277-6_4.

[39] Z. Fan, H. Ruan, S. Mechtaev, and A. Roychoudhury, "Oracle-Guided Program Selection from Large Language Models," *ISSTA*, 2024, doi: 10.1145/3650212.3680308.

[40] A. K. Vajjala *et al.*, "Enhancing Differential Testing: LLM-Powered Automation in Release Engineering," *IEEE/ACM International Conference on Software Engineering - Software Engineering in Practice*, 2025, doi: 10.1109/icse-seip66354.2025.00059.

[41] Z. Liu, H. Wang, T. Xu, and B. Wang, "RAG-Driven multiple assertions generation with large language models," *Empirical Software Engineering*, 2025, doi: 10.1007/s10664-025-10641-1.

[42] C. Ni, X. Wang, X. Yin, L. Chen, and G. Ma, "A Cascaded Pipeline for Self-Directed, Model-Agnostic Unit Test Generation via LLMs," *Proc. International Symposium on Software Reliability Engineering, ISSRE*, 2025, doi: 10.1109/issre66568.2025.00037.

[43] J. C. Alonso, A. Martin-Lopez, S. Segura, G. Bavota, and A. Ruiz-Cortés, "SATORI: Static Test Oracle Generation for REST APIs," *Proc. 40th IEEE/ACM International Conference on Automated Software Engineering, ASE*, 2025, doi: 10.1109/ase63991.2025.00116.

[44] J. Shin, H. Hemmati, M. Wei, and S. Wang, "Assessing Evaluation Metrics for Neural Test Oracle Generation," *IEEE Transactions on Software Engineering*, 2024, doi: 10.1109/tse.2024.3433463.

[45] K. Etemadi, M. Sirjani, M. Helali Moghadam, P. Strandberg, and P. Pettersson, "LLM-Based Property-Based Test Generation for Guardrailing Cyber-Physical Systems," *Lecture Notes in Computer Science*, 2026, doi: 10.1007/978-3-032-07132-3_3.

[46] S. R. R. dos Santos, M. S. Soares, R. E. S. Fernandes, F. G. Rocha, M. C. B. dos Santos, and S. Marczak, "Increasing Test Coverage by Automating BDD Tests in Proofs of Concepts (POCs) using LLM," *Brazilian Symposium on Software Quality (SBQS)*, 2024, doi: 10.1145/3701625.3701637.

[47] S. Y. Shin, F. Pastore, D. Bianculli, and A. Baicoianu, "Towards Generating Executable Metamorphic Relations Using Large Language Models," *Communications in Computer and Information Science*, 2024, doi: 10.1007/978-3-031-70245-7_9.

[48] H. Lee, G. An, and S. Yoo, "Metamon: Finding Inconsistencies between Program Documentation and Behavior using Metamorphic LLM Queries," *Proc. IEEE/ACM International Workshop on Large Language Models for Code, LLM4Code*, 2025, doi: 10.1109/llm4code66737.2025.00020.

[49] S. R. R. dos Santos *et al.*, "Automated Test Generation Using LLM Based on BDD: A Comparative Study," *International Conference on Web Information Systems and Technologies, WEBIST - Proceedings*, 2025, doi: 10.5220/0013683600003985.

[50] C. Hazott and D. Große, "LLM-assisted Metamorphic Testing of Embedded Graphics Libraries," *Forum on Specification and Design Languages*, 2025, doi: 10.1109/fdl68117.2025.11165405.

[51] P. C. Cañizares, P. Gómez-Abajo, E. Guerra, and J. de Lara, "Towards metamorphic testing with LLM-based workflows: Metamorphic relation inference and follow-up test case generation," *Information and Software Technology*, vol. 196, art. 108150, 2026, doi: 10.1016/j.infsof.2026.108150.

[52] H. Tanaka, H. Tanaka, K. Shimari, and K. Matsumoto, "Understanding the Characteristics of LLM-Generated Property-Based Tests in Exploring Edge Cases," *Proc. 2nd IEEE/ACM International Conference on AI-powered Software, AIware*, 2025, doi: 10.1109/aiware69974.2025.00009.

[53] J. Jagielski, C. Rojas, and M. Abel, "Exploratory Study on Private GPTs for LLM-Driven Test Generation in Software and Machine Learning Development," *Proc. International Generative AI and Computational Language Modelling Conference (GACLM)*, 2025, doi: 10.1109/gaclm67198.2025.11232354.

[54] Y. Zhang, D. Towey, and M. Pike, "Automated Metamorphic-Relation Generation with ChatGPT: An Experience Report," *Proc. International Computer Software and Applications Conference*, 2023, doi: 10.1109/compsac57700.2023.00275.

[55] J. Johnson, J. Mahmud, O. Chaparro, K. Moran, and M. Fazzini, "Generating Failure-Based Oracles to Support Testing of Reported Bugs in Android Apps," *Proc. 40th IEEE/ACM International Conference on Automated Software Engineering, ASE*, 2025, doi: 10.1109/ase63991.2025.00182.

[56] Y. Zhang, T. Y. Chen, M. Pike, D. Towey, Z. Ying, and Z. Q. Zhou, "Enhancing autonomous driving simulations: A hybrid metamorphic testing framework with metamorphic relations generated by GPT," *Information and Software Technology*, vol. 187, art. 107828, 2025, doi: 10.1016/j.infsof.2025.107828.

[57] Q. Zhang *et al.*, "Exploring Automated Assertion Generation via Large Language Models," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 3, art. 81, pp. 1–25, 2025, doi: 10.1145/3699598.

[58] I. Hayet, A. Scott, and M. D'Amorim, "ChatAssert: LLM-Based Test Oracle Generation With External Tools Assistance," *IEEE Transactions on Software Engineering*, vol. 51, no. 1, pp. 305–319, 2025, doi: 10.1109/tse.2024.3519159.

[59] J. Quispe, H. Cordova, and L. Wong, "Approach for Generating and Executing Acceptance Tests Based on Computer Vision and GPT-4," *Proc. International Conference on Software Engineering and Computer Science (CSECS)*, 2026, doi: 10.1109/csecs69124.2026.11541611.

[60] G. E. Raptis, M. T. Khan, C. Koulamas, and D. Serpanos, "Synthesizing Inline Security Monitors for ICS Using Generative AI and FormalBench," *IECON*, 2025, doi: 10.1109/iecon58223.2025.11221437.

[61] G. E. Raptis, M. T. Khan, C. Koulamas, and D. Serpanos, "LLM-based Generation of Formal Specification for Run-time Security Monitoring of ICS," *Proc. IEEE International Conference on Cyber Security and Resilience (CSR)*, 2025, doi: 10.1109/csr64739.2025.11130130.

[62] Q. Zhang *et al.*, "Improving Retrieval-Augmented Deep Assertion Generation via Joint Training," *IEEE Transactions on Software Engineering*, vol. 51, no. 4, pp. 1232–1247, 2025, doi: 10.1109/tse.2025.3545970.

[63] Q. Liang, C. Li, F. Yang, and Q. Xu, "A Detection Method for Text Input-Induced Non-Crashing Functional Bugs in Mobile GUI Testing," *Proc. International Conference on Advanced Electronic Technology, Computers and Software Engineering (AETCSE)*, 2026, doi: 10.1109/aetcse69203.2026.11504024.

[64] Y. Shang, Q. Zhang, C. Fang, S. Gu, J. Zhou, and Z. Chen, "A Large-Scale Empirical Study on Fine-Tuning Large Language Models for Unit Testing," *Proc. ACM Softw. Eng.*, 2025, doi: 10.1145/3728951.

[65] X. Ma, J. Chen, L. Luo, and Q. Zeng, "LLM-Assisted IoT Testing: Finding Conformance Bugs in Matter SDKs," *Proc. 31st Annual International Conference on Mobile Computing and Networking*, 2025, doi: 10.1145/3680207.3765257.

[66] Q. Xu, G. Wang, L. Briand, and K. Liu, "Hallucination to Consensus: Multi-Agent LLMs for End-to-End JUnit Test Generation," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3803418.

[67] Y. Li, Y. Ji, X. Chen, Z. Shen, J. Ge, and B. Luo, "FURINA: A Multi-Agent Framework for Automated Patch Correctness Assessment," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3811825.

[68] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri, "TOGA: a neural method for test oracle generation," *Proc. International Conference on Software Engineering (ICSE)*, 2022, doi: 10.1145/3510003.3510141.

[69] Z. Liu, K. Liu, X. Xia, and X. Yang, "Towards More Realistic Evaluation for Neural Test Oracle Generation," *Proc. ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, doi: 10.1145/3597926.3598080.

[70] S. B. Hossain, A. Filieri, M. B. Dwyer, S. Elbaum, and W. Visser, "Neural-Based Test Oracle Generation: A Large-Scale Evaluation and Lessons Learned," *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023, doi: 10.1145/3611643.3616265.

[71] C. Xu, H. Zhu, S. Chen, J. Wu, V. Terragni, and S. C. Cheung, "MR-Coupler: Automated Metamorphic Test Generation via Functional Coupling Analysis," *Proc. ACM Softw. Eng.*, 2026, doi: 10.1145/3808213.

[72] A. Celik and Q. H. Mahmoud, "GEM: A Framework for Strengthening LLM-Generated Unit Tests Using Mutation Feedback," *Proc. Conferencia Iberoamericana de Software Engineering (CIbSE)*, 2026, doi: 10.5753/cibse.2026.42441.

[73] Y. He, J. Huang, H. Yu, and T. Xie, "An Empirical Study on Focal Methods in Deep-Learning-Based Approaches for Assertion Generation," *Proc. ACM Softw. Eng.*, 2024, doi: 10.1145/3660785.

[74] C. Li *et al.*, "AutoOracle: High-Quality C++ Test Oracle Generation via Data Quality-Driven and Filtering-Enabled LLMs," *Proc. IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, doi: 10.1145/3786583.3786868.

[75] A. Balestra, A. Nolasco, F. Molina, D. Garbervetsky, R. Degiovanni, and N. Aguirre, "Improving Dynamic Specification Inference with LLM-Generated Counterexamples," *Proc. IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2026, doi: 10.1109/ICST69053.2026.00059.

[76] X. Teoh, Y. Lin, D. M. Nguyen, R. Ren, W. Zhang, and J. S. Dong, "WebTestPilot: Agentic End-to-End Web Testing against Natural Language Specification by Inferring Oracles with Symbolized GUI Elements," *Proc. ACM Softw. Eng.*, 2026, doi: 10.1145/3797115.

[77] D. Molinelli, A. Martin-Lopez, E. Zackrone, B. Eken, M. D. Ernst, and M. Pezzè, "Tratto: A Neuro-Symbolic Approach to Deriving Axiomatic Test Oracles," *Proc. ACM Softw. Eng.*, 2025, doi: 10.1145/3728960.

[78] Y. Hu *et al.*, "Minuku: Detecting Diverse Display Issues in Mobile Apps with Small-scale Dataset," *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, doi: 10.1109/ASE63991.2025.00270.

[79] C. Wen *et al.*, "Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification," *Lecture Notes in Computer Science*, 2024, doi: 10.1007/978-3-031-65630-9_16.

[80] H. Taherkhani, A. DaghighFarsoodeh, M. Chowdhury, H. V. Pham, and H. Hemmati, "Consistency Meets Verification: Enhancing Test Generation Quality in Large Language Models Without Ground-Truth Solutions," *Proc. IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2026, doi: 10.1109/ICST69053.2026.00039.

[81] T. Kiecker, J. A. Sparka, M. Reuter, A. Ziegler, and L. Grunske, "CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation," *Proc. ACM Softw. Eng.*, 2026, doi: 10.1145/3808175.

[82] M. Becker *et al.*, "Just-in-Time Catching Test Generation at Meta," *Proc. ACM International Conference on the Foundations of Software Engineering, Companion (FSE Companion)*, 2026, doi: 10.1145/3803437.3805202.

[83] G. Zhang and J. Zhai, "POSTCONDBENCH: Benchmarking Correctness and Completeness in Formal Postcondition Inference," *Findings of the Association for Computational Linguistics (ACL Findings)*, 2026, doi: 10.18653/v1/2026.findings-acl.1768.

[84] C. C. Le *et al.*, "SpecMind: Cognitively Inspired, Interactive Multi-Turn Framework for Postcondition Inference," *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026, doi: 10.48550/arXiv.2602.20610.

[85] S. Ruberto, J. Perera, G. Jahangirova, and V. Terragni, "From Implemented to Expected Behaviors: Leveraging Regression Oracles for Non-regression Fault Detection using LLMs," *Proc. IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2025, doi: 10.1109/ICSTW64639.2025.10962503.

[86] H. Li *et al.*, "Retrieval-Augmented Fine-Tuning for Improving Retrieve-and-Edit Based Assertion Generation," *IEEE Transactions on Software Engineering*, 2025, doi: 10.1109/TSE.2025.3558403.

[87] Q. Zhang *et al.*, "Improving Deep Assertion Generation via Fine-Tuning Retrieval-Augmented Pre-trained Language Models," *ACM Transactions on Software Engineering and Methodology*, 2025, doi: 10.1145/3721128.

[88] Z. Wang *et al.*, "A Tale of 1001 LoC: Potential Runtime Error-Guided Specification Synthesis for Verifying Large-Scale Programs," *Proc. ACM Program. Lang.*, 2026, doi: 10.1145/3798268.

[89] G. Sakkas, P. Sahu, K. Ong, and R. Jhala, "Neurosymbolic Modular Refinement Type Inference," *Proc. International Conference on Software Engineering (ICSE)*, 2025, doi: 10.1109/ICSE55347.2025.00090.

[90] Y. W. Zhang, Z. Jin, Z. J. Wang, Y. Xing, and G. Li, "SAGA: Summarization-Guided Assert Statement Generation," *Journal of Computer Science and Technology*, 2025, doi: 10.1007/s11390-023-2878-6.

[91] G. E. Raptis, M. T. Khan, C. Koulamas, and D. Serpanos, "Cross-Model Evaluation of LLMs for Generating Formal Specification of Distributed Industrial Control Systems," *Procedia Computer Science*, 2025, doi: 10.1016/j.procs.2025.10.053.

[92] H. Taherkhani, J. Shin, M. A. Tahir, M. R. H. Misu, V. S. Gattani, and H. Hemmati, "Toward Automated Validation of Language Model Synthesized Test Cases Using Semantic Entropy," *IEEE Transactions on Software Engineering*, 2026, doi: 10.1109/TSE.2026.3664287.

[93] N. Mündler, M. Müller, J. He, and M. Vechev, "SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents," *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, doi: 10.52202/079017-2601.

[94] K. Liu *et al.*, "LLM-Powered Test Case Generation for Detecting Bugs in Plausible Programs," *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, doi: 10.18653/v1/2025.acl-long.20.

[95] S. Alagarsamy, C. Tantithamthavorn, and A. Aleti, "A3Test: Assertion-Augmented Automated Test case generation," *Information and Software Technology*, 2024, doi: 10.1016/j.infsof.2024.107565.

[96] K. Pei, D. Bieber, K. Shi, C. Sutton, and P. Yin, "Can Large Language Models Reason about Program Invariants?," *Proc. International Conference on Machine Learning (ICML), PMLR 202*, 2023, pp. 27496–27520. [Online]. Available: https://proceedings.mlr.press/v202/pei23a.html.

[97] V. I. T. Perera and D. I. De Silva, "Evaluating Large Language Models for Software Testing: A Systematic Review of Metrics and Practices," *Lecture Notes in Networks and Systems*, 2026, doi: 10.1007/978-3-032-18913-4_27.

[98] F. Molina, A. Gorla, and M. D'Amorim, "Test Oracle Automation in the Era of LLMs," *ACM Transactions on Software Engineering and Methodology*, 2025, doi: 10.1145/3715107.

[99] A. Duque-Torres, S. Fischer, and C. Klammer, "Metamorphic Testing for Sequential Prediction Models: A Survey of LSTMs and LLMs," *Proc. IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion, SANER-C*, 2026, doi: 10.1109/saner-c67878.2026.00054.

[100] J. Zhang, X. Hu, C. Gao, X. Xia, and S. Li, "Enhancing Automated Unit Test Generation with Large Language Models: A Systematic Literature Review," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3802827.

[101] S. Fischer and W. Kloihofer, "LLM Agents for Autonomous System Testing: A Semi-structured Literature Review," *Lecture Notes in Business Information Processing*, 2026, doi: 10.1007/978-3-032-24216-7_9.

[102] I. Pysmennyi, R. Kyslyi, and K. Kleshch, "AI-DRIVEN TOOLS IN MODERN SOFTWARE QUALITY ASSURANCE: AN ASSESSMENT OF BENEFITS, CHALLENGES, AND FUTURE DIRECTIONS," *Technology Audit and Production Reserves*, 2025, doi: 10.15587/2706-5448.2025.330595.

[103] Umama *et al.*, "LLM-Based Code Generation: A Systematic Literature Review With Technical and Demographic Insights," *IEEE Access*, 2025, doi: 10.1109/access.2025.3631952.

[104] Q. Wang, J. Wang, M. Li, Y. Wang, and Z. Liu, "A Roadmap for Software Testing in Open-Collaborative and AI-Powered Era," *ACM Trans. Softw. Eng. Methodol.*, 2025, doi: 10.1145/3709355.

[105] A. J. Calderón *et al.*, "On the Use of Artificial Intelligence in Software Testing: State of the Art and Application to Satellite Systems," *ACM Trans. Cyber-Phys. Syst.*, 2026, doi: 10.1145/3783988.

[106] D. Amalfitano, R. Coppola, D. Distante, and F. Ricca, "AI in GUI-based testing: A survey of techniques, tools, and perceived advantages and limitations," *Journal of Systems and Software*, 2026, doi: 10.1016/j.jss.2025.112751.

[107] P. Sweetser, "Large Language Models and Video Games: A Preliminary Scoping Review," *Proc. 6th ACM Conference on Conversational User Interfaces*, 2024, doi: 10.1145/3640794.3665582.

[108] Q. Song, H. Ye, M. Harman, and F. Sarro, "Generative AI for Testing of Autonomous Driving Systems: A Survey," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3806653.



Ali Hassaan Mughal (Graduate Student Member, IEEE) has worked as a Senior Software Developer and Team Lead at Xpressdocs, and earlier at Paycom and a stealth robotics company. His work at Xpressdocs centered on software testing and quality assurance, including the design and evaluation of LLM-based multi-agent systems. He is pursuing an Applied MBA in Data Analytics at Texas Wesleyan University, USA, and holds an M.Sc. in Computer Science from Kansas State University. His research interests include automated software testing, web application quality assurance, applied machine learning, LLMs, and agentic AI systems. Earlier he was a graduate teaching and research assistant at Kansas State University, where he taught core computer science courses and worked on deep learning for spacecraft trajectory prediction, and a research intern at Aalto University, Finland, on a human computer interaction project. He has published in Elsevier, IEEE, and EAI venues. Since 2024 he has served as a reviewer for Expert Systems with Applications, Computers and Electrical Engineering, the International Journal of Information Management, and the ACM Conference on Computer-Supported Cooperative Work and Social Computing (CSCW).



Muhammad Bilal is an AI and Digitalization Consultant in the German industrial sector. He holds a Master of Science in Management from the Technical University of Munich, Germany, and has previously worked as a Software Engineer, Business Analyst, and Product Owner. His research interests include the impact of technology on business performance, product quality analytics, the automation of industrial pipelines, LLMs, and agentic AI systems. His career spans product ownership at Persivia, a clinical AI company; business development at NVision Quantum Technologies, a quantum biotechnology company; healthcare software development at Cognitive Healthcare International; and research at LMU Munich. His current technical work is in agentic systems and LLMs, covering retrieval-augmented generation, agent orchestration, structured output validation, and the evaluation and grounding verification that decide whether such systems can be trusted. He has also published on verifying production LLM applications. His master's thesis and subsequent research examine the sustainability and economics of AI, particularly how its energy use relates to its economic value. He is the corresponding author.